

A Unified Framework for Hybrid Network Intrusion Detection

Yan Wang^{1b}, Sheng Cao^{1b}, *Member, IEEE*, Jingwei Li^{1b}, *Member, IEEE*,
and Xiaosong Zhang^{1b}, *Senior Member, IEEE*

Abstract—Lately, hybrid network intrusion detection systems (HNIDSs) have progressed significantly. Through the cascade or ensemble of multiple machine learning models, HNIDS benefits from each model and achieves better performance. A widely adopted framework for designing HNIDS consists of two models: a misuse detector and an anomaly detector. However, (1) benign traffic must be analyzed by both models, reducing inference speed; (2) the misuse detector performs dual functionalities, leading to suboptimal accuracy; (3) deploying the misuse and anomaly detectors on two devices introduces substantial latency and restricts distributed deployment. In this article, we propose a unified framework called AUF. To solve (1), we deploy the anomaly detector in the first stage rather than the second, which improves inference speed. To solve (2), we employ two independent models to implement the misuse detector’s functionality, enhancing overall accuracy. To solve (3), we ensure that the different models operate independently, supporting distributed deployment. To demonstrate the effectiveness of the AUF framework, we implement XGBoost for detection and classification and propose an adaptive k -nearest neighborhood-based approach to achieve accurate discrimination. We also introduce zero-shot learning to showcase the framework’s customized model. Extensive experiments validate the effectiveness of the AUF framework and methods. Our code is available at <https://github.com/wangyann2000/A-Unified-Framework-for-Hybrid-Network-Intrusion-Detection>.

Index Terms—Intrusion detection, anomaly detection, out-of-distribution detection, multi-class classification, zero-shot learning, network security.

I. INTRODUCTION

THE NETWORK intrusion detection system (NIDS) is a security tool that monitors the network [1], [2]. It

aims to detect and respond to malicious behavior or security incidents promptly. Its primary function is to detect malicious network traffic and provide relevant information for network administrators to take defensive or responsive measures. Most NIDS construct intrusion detection models through machine learning or deep learning, converting network intrusion detection into anomaly detection or multi-class classification tasks [3], [4], [5].

Due to the increasing types of malicious traffic, relying on a single machine learning model to accurately detect malicious traffic is difficult [6], [7], [8]. Therefore, some studies attempted to integrate multiple machine-learning models to enhance the performance of intrusion detection, referred to as the HNIDS [9], [10], [11]. The core idea of HNIDS is to cascade multiple models and fully exploit their advantages. However, due to the guidance of the framework, i.e., specifying the functionalities required for each model and their correlations, many HNIDSs simply cascade multiple models together, as shown in Figure 1(a). This Non-framework cascading tends to focus on improving the accuracy of the hybrid model. However, they ignore some of the other important performance metrics, including detection accuracy, functionality completeness (e.g., detecting malicious traffic and identifying malicious traffic classes), inference speed, and distributed deployment capability. For convenience, we name such an issue the “loss-gain imbalance” in HNIDS.

Proposing the framework for guiding the design of HNIDS is the key to alleviating the “loss-gain imbalance,” where Kim et al. [12] construct the framework that combines a misuse and an anomaly detector (MADF), as shown in Figure 1(b). All traffic is first analyzed by the misuse detector, followed by the anomaly detector for the MADF framework. The misuse detector identifies seen malicious traffic, i.e., types/classes of malicious traffic available during training based on comparing features stored in the database, while the anomaly detector identifies unseen malicious traffic, i.e., new types/classes of malicious traffic unseen during training by detecting features that deviate from benign traffic. Traffic that successfully passes the detection of both models will be considered benign traffic. Despite the MADF framework’s attempt, it still suffers from the “loss-gain imbalance” issue: (1) benign traffic that constitutes most of the network traffic needs to pass through both models, resulting in low inference speed; (2) the misuse detector both differentiates between seen and unseen malicious traffic and classifies seen malicious traffic, leading to suboptimal accuracy; (3) the misuse and

Received 29 September 2024; revised 5 March 2025 and 13 July 2025; accepted 11 September 2025. Date of publication 16 September 2025; date of current version 5 December 2025. This work is supported by Key Research and Development Program of Sichuan Province (2023ZHJY0006), Sichuan Province Science and Technology Support Program (2024NSFSC0004), and National Natural Science Foundation of China (U2336204). The associate editor coordinating the review of this article and approving it for publication was R. Doriguzzi-Corin. (*Corresponding author: Sheng Cao.*)

Yan Wang and Jingwei Li are with the School of Computer Science and Engineering (School of Cyber Security), University of Electronic Science and Technology of China, Chengdu 610054, Sichuan, China.

Sheng Cao is with the School of Computer Science and Engineering (School of Cyber Security), the Yibin Park, the Sichuan-Chongqing Co-construction Key Laboratory of Digital Economy Intelligence and Security, and the Blockchain Storage and Trade Engineering Center of Sichuan province, University of Electronic Science and Technology of China, Chengdu 610054, Sichuan, China (e-mail: caosheng@uestc.edu.cn).

Xiaosong Zhang is with the School of Computer Science and Engineering (School of Cyber Security) and the Shenzhen Institute for Advanced Study, University of Electronic Science and Technology of China, Chengdu 610054, Sichuan, China.

Digital Object Identifier 10.1109/TNSM.2025.3609854

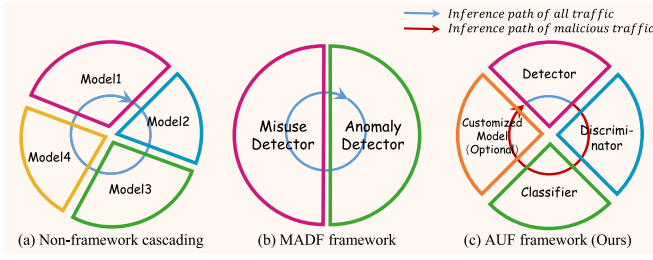


Fig. 1. (a) Visualization of the “Non-framework cascading”: Different methods introduce models with different functionalities, and the relationships between models vary. All network traffic typically needs to pass through all models for analysis. (b) Visualization of the MADF framework: Two models are responsible for separately detecting seen and unseen malicious traffic, requiring all traffic to be analyzed by both models. (c) Visualization of the AUF framework: Most network traffic only needs to be analyzed by a single detector. The models are independent in functionality and support distributed deployment.

anomaly detectors deployed across different devices lead to high traffic transmission costs and introduce substantial latency, making distributed deployment difficult.

To tackle the “loss-gain imbalance,” we propose a unified framework for HNIDS called AUF. As shown in Figure 1(c), the AUF framework consists of four unified, decoupled, cascaded models: a detector, a discriminator, a classifier, and a customized model. The detector distinguishes between benign and malicious traffic, including seen and unseen malicious traffic. The discriminator differentiates between seen and unseen detected malicious traffic and forwards seen malicious traffic to the classifier. The classifier categorizes detected seen malicious traffic into its corresponding classes. The customized model can be integrated in a cascaded manner to analyze malicious traffic based on the requirements. Compared to the MADF framework, the advantages of the AUF framework are threefold. Firstly, benign traffic accounts for most of the network traffic and is analyzed only by the detector, thereby improving inference speed. Furthermore, each model focuses on implementing a single functionality, leading to better performance than models handling multiple functionalities. Finally, the decoupled models support distributed deployment, such as cloud-edge collaborative deployment. The detector can be deployed on edge since malicious traffic detection requires high real-time performance. In contrast, the discriminator, classifier, and customized models can be deployed in the cloud to reduce the computational burden on edge devices.

We further introduce different methods to showcase the design of HNIDS based on the AUF framework. (1) Detector and Classifier: We adopt tree-based models adaptable in multiple scenarios as the detector and the classifier. (2) Discriminator: We propose an adaptive k -nearest neighborhood-based (KNN) approach for better discrimination. The key of the adaptive KNN-based approach is to determine whether malicious traffic belongs to the seen or unseen based on the number of neighbors from the seen categories in k -nearest neighbors. The value of k for each sample is adaptively determined according to its local density, mitigating the impact of class imbalance. (3) Customized model: We introduce zero-shot learning (ZSL) to assist administrators in labeling unseen malicious traffic. ZSL aims to

identify the categories of unseen samples with the help of auxiliary information. With the assistance of the classifier trained based on ZSL, administrators only need to provide auxiliary information and validate the classification results, thereby reducing labeling costs. The proposed semantic-guided zero-shot learning approach (SG-ZSL) enhances the correlation between auxiliary information and features by maximizing mutual information, thus extracting more discriminative features for unseen malicious traffic. We summarize our contributions as follows:

- **Problem and Framework.** We address the limitations of “Non-framework cascading” and the MADF framework—the “loss-gain imbalance” issue in HNIDS—by proposing a unified framework AUF. The AUF framework cascades four unified models: a detector, a discriminator, a classifier, and a customized model. HNIDS built upon the AUF will benefit from the unified framework’s advantage, with faster inference speed, better overall accuracy, and distributed deployment capabilities.
- **Discriminator.** The adaptive KNN-based approach exploits the distribution differences between seen and unseen malicious traffic in nearest neighbors. The adaptive k estimated based on local density benefits minority classes, thereby achieving accurate discrimination.
- **Customized Model.** We introduce ZSL to train a classifier as the customized model within the AUF framework to reduce the labeling cost for unseen malicious traffic. The semantic-guided and contrastive learning loss preserves semantic consistency and enhances the discriminability of malicious traffic features.
- **Sufficient Experiment Support.** Our experimental results on the CIC-IDS2017 and the BoT-IoT datasets demonstrate that the AUF framework (i) achieves satisfying performance both in individual models and in the cascaded hybrid model, (ii) highlights the importance of handling class imbalance and generalization to unseen classes, and (iii) provides new baselines and insights for future research.

II. RELATED WORK

A. HNIDS Built Upon “Non-Framework Cascading”

Table I illustrates the differences in functionalities, inference speed, and distributed deployment support between SOTA HNIDS, where N, M, and U denote HNIDS built upon the “Non-framework cascading,” the MADF framework and the proposed unified framework, respectively. Based on model functionality, we categorize the models in HNIDS into detector (anomaly detector), discriminator, classifier (misuse detector), and the customized model. Detector refers to models built on anomaly detection techniques, characterized by their ability to distinguish between benign and malicious traffic, regardless of whether the malicious traffic categories were seen or unseen during training. Discriminator refers to models constructed based on out-of-distribution detection techniques, capable of differentiating between seen and unseen malicious

TABLE I
FRAMEWORK COMPARISON

Method	Frame- work	Models/Algorithms ¹				Steps ²	Distributed Deployment	Open- Source
		Detector	Discriminator	Classifier	Customized Model			
ElSayed et al. [13]	N			1	1	2		
Mushtaq et al. [14]	N			1	1	2		
Lan et al. [15]	N	1			1	2		
de Souza et al. [16]	N			2		1	✓	
Chen et al. [17]	N			3		2	✓	
An et al. [18]	N	1			1	2		
Bovenzi et al. [19]	N	1		1		1	✓	
Verkerken et al. [20]	N	1		1	1	2		✓
Kye et al. [21]	N	2				2		
Alzaqebah et al. [22]	N			Multiple		Multiple		
Wei et al. [23]	N			2		2		
Yang et al. [24]	M	1		1		2		
Li et al. [25]	M	1		1		2		
Yang et al. [26]	M	1		1		2		✓
Chen et al. [27]	M	Multiple		1		2		
Zhong et al. [28]	M	1		1		2		
Ours	U	1	1	1	1	1	✓	✓

¹ Number of detectors, discriminators, classifiers or customized models. Blank indicates the method does not include corresponding models/algorithms.

² Number of models for analyzing the benign traffic.

traffic. Classifier refers to binary or multi-class models. Binary classifiers can distinguish between benign and seen malicious traffic, while multi-class classifiers can identify categories of seen malicious traffic. The customized model have no fixed functionality.

ElSayed et al. (JNCA 2021) [13], Mushtaq et al. (Appl. Soft Comput. 2022) [14], Lan et al. (Appl. Intell. 2023) [15], emphasize the importance of discriminative feature extraction in enhancing accuracy, designing HNIDSs that include both feature extractors and classifiers. These three approaches employed convolutional neural networks, autoencoders, and triplet networks as feature extractors to capture discriminative features. ElSayed's approach, which uses a single machine learning-based classifier, can only perform binary or multi-class classification separately without the ability to detect seen malicious traffic types and unseen malicious traffic simultaneously. Mushtaq's method leverages long short-term memory (LSTM) networks for classification, which can only detect seen malicious traffic. Lan's approach, based on one-class support vector machines, can detect both seen and unseen malicious traffic but cannot identify specific categories of seen malicious traffic.

Some studies aim to develop HNIDS with more complete functionalities by cascading multiple models, enabling both the detection of unseen malicious traffic and the classification of seen malicious traffic. For instance, de Souza et al. (CN 2020) [16] cascade a binary classifier with a multi-class classifier, where the binary classifier distinguishes between benign and malicious traffic, and the multi-class classifier identifies the specific category of malicious traffic. Similar to Cristiano Antonio et al., Chen et al. (CN 2024) [17] further integrate an

additional multi-class classifier to enhance accuracy. However, neither of these approaches can detect unseen categories of malicious traffic. An et al. (TCCN 2022) [18] introduced the Birch clustering algorithm and an autoencoder to detect malicious traffic. Bovenzi et al. [19] design the Multi-Modal Deep Auto-Encoder (M2-DAE) to detect both seen and unseen malicious traffic and employ the classifier's output to determine the category of malicious traffic. Verkerken et al. [20] extend Bovenzi's hybrid system by introducing an extension stage for recalling wrongly detected benign traffic. However, these three methods rely on unsupervised learning for detection, making it difficult to maintain low false negative and false positive rates.

Kye et al. (SPL 2022) [21] and Alzaqebah et al. (COSE 2023) [22] apply multiple models directly to malicious traffic detection. Kye et al. develop a HNIDS composed of an encoder and a decoder. The encoder first calculates the Mahalanobis distance as an anomaly score to assess the deviation of the input traffic from benign traffic, setting a low threshold to detect traffic with high anomaly scores. Traffic that passes through the encoder is forwarded into the decoder for secondary detection, and traffic with high reconstruction loss is flagged as malicious. Alzaqebah et al. decompose the multi-class classification task into several binary classification tasks. Each base classifier focuses on distinguishing between benign traffic and a specific type of malicious traffic, and final predictions are jointly made based on each classifier's prediction. These methods cannot detect unseen malicious traffic, and the coupled design restricts the distributed deployment of machine-learning models across different devices.

B. HNIDS Built Upon the “MADF” Framework

HNIDS built upon the MADF framework typically includes a misuse and an anomaly detector, enabling the detection of seen and unseen malicious traffic and the recognition of malicious traffic categories. For instance, Yang et al. (TIFS 2021) [24] leveraged the conditional VAE as the misuse detection phase to determine whether the input traffic belongs to benign or specific seen malicious classes and utilized the generative-based anomaly detector to identify unseen malicious traffic. Yang et al. (ESWA 2024) [26] proposed an architecture that employed a stacking-based classifier as the misuse detector and utilized the generative-based anomaly detector for detecting unseen malicious traffic. Li et al. (IOT 2022) [25] followed the same misuse detector but differed in that the anomaly detector employed the clustering labeling k-means (CL k-means) for unseen malicious traffic detection. Some studies route all traffic through the anomaly detector to address the issue where some unseen malicious traffic is misidentified as seen malicious traffic by the misuse detector. Chen et al. [27] (CN 2024) develop a semi-supervised learning-based approach, constructing a multi-class classifier (identifying K seen malicious traffic classes) and K binary classifiers (each corresponding to a seen class, distinguishing it from unseen traffic). Each binary classifier focuses on determining whether samples predicted as belonging to a specific class are unseen malicious traffic. During prediction, traffic is first analyzed and classified into a specific category by the multi-class classifier and then forwarded to the corresponding binary classifier to determine if it is unseen malicious traffic. Zhong et al. [28] (TIFS 2024) build the misuse detector based on deep neural networks and contrastive learning, employing a deep k-nearest neighbors algorithm to determine whether traffic belongs to unseen malicious categories. The deep k-nearest neighbors algorithm takes hidden layer features from both the deep neural network and the adversarial neural network as input to more accurately identify unseen malicious traffic. Although the MADF framework integrates functionalities including detection, discrimination, and classification, it still encounters the “loss-gain imbalance” issue. In contrast, the proposed AUF framework achieves “unification” across three key dimensions and effectively addresses the loss-gain imbalance problem that previous approaches struggle to overcome: (1) Unification of models and corresponding functionalities. As mentioned above, in MADF-based and certain cascaded HNIDS architectures, individual models are responsible for multiple tasks simultaneously. For instance, classifiers in [19] and [20] are required to both distinguish between seen and unseen malicious traffic and identify the categories of seen malicious traffic. In contrast, AUF adopts a functionally decoupled design where each model is dedicated to a single task. This design enables more comprehensive functionalities, faster inference speed for benign traffic, and improved accuracy. (2) Unification of modeling and traffic processing. Some existing HNIDSs restrict the way of constructing models. For example, [19] and [20] specify in their architectures that anomaly detectors be trained using only benign traffic. In contrast, AUF imposes no such restrictions.

Each module within AUF (detector, discriminator, classifier, or customized model) can be implemented using unsupervised, semi-supervised, or supervised learning methods, and can accept either raw traffic or preprocessed features as input. This flexibility not only ensures compatibility with a wide range of existing NIDS methods but also provides a scalable and extensible foundation for future developments in intrusion detection research. (3) Unification of open-source code. Due to the lack of publicly available source code in most existing hybrid NIDS implementations, reproducing prior work is often difficult and unreliable. As shown in Table I, we publicly release the source code of AUF, enabling future work to build upon the proposed AUF framework.

C. Contrastive Embedding-Based Zero-Shot Learning

Classifying unseen malicious traffic is challenging due to the lack of labeled samples for training. ZSL is a machine learning method that enables models to infer when facing unseen classes or samples by leveraging auxiliary semantic information and class relationships. The key of ZSL lies in constructing a “bridge” between features and semantic information. Many methods have introduced contrastive embeddings into ZSL in recent years and achieved promising results. Han et al. [30] first introduced contrastive embeddings into ZSL. The model learns contrastive embeddings at both class and instance levels, assisting generative models in generating more discriminative features for unseen classes. Building upon [30], Fan et al. [32] further decomposed embeddings into semantically related and semantically unrelated embeddings. Li et al. [31] introduced feature contrastive optimization module (FCOM) and cycle reconstruction loss in the proposed method, encouraging the model to generate visually consistent and discriminative features with semantic coherence. Wang et al. [33] utilized generated samples to train the contrastive embedding model for more generalized embedding. Kong et al. [34] proposed an Intra-Class Compactness Enhancement method (ICCE) to enhance intra-class compactness, jointly promoting intra-class compactness and inter-class separability in embedding and visual feature space.

III. METHODOLOGY

A. Overall Architecture

As illustrated in Figure 2, the AUF framework consists of four cascaded stages. Given a feature set $X = \{x_1^b, \dots, x_n^b, x_{n+1}^a, \dots, x_{n+m}^a\}$ composed of n benign traffic samples and m malicious traffic samples as input, the detector at stage 1 outputs anomaly scores for each sample. If the anomaly score exceeds τ_1 , the sample is recognized as malicious traffic and forwarded to the discriminator at stage 2. Given the detector has detected d seen malicious traffic samples and $m - d$ unseen malicious samples forming a feature set $X_a = \{x_1^s, \dots, x_d^s, x_{d+1}^u, \dots, x_m^u\}$, the discriminator at stage 2 outputs out-of-distribution (OOD) scores for each detected malicious traffic sample. If the OOD score exceeds τ_2 , the sample is considered unseen malicious

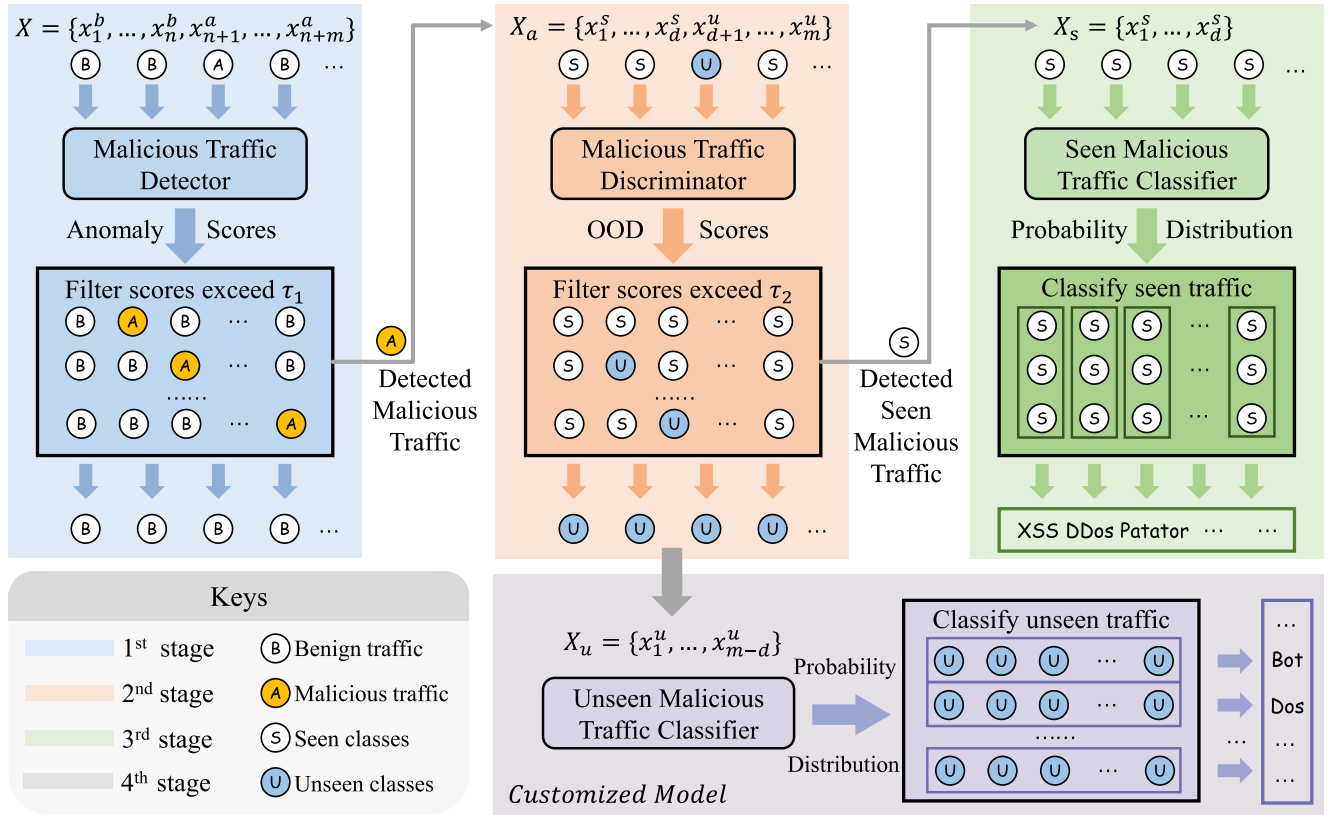


Fig. 2. Proposed AUF Framework. Blue, orange, green, and purple colors indicate the corresponding stages of the detector, discriminator, and classifier with the customized model. It integrates a detector, discriminator, classifier, and customized model in a cascaded manner, with each model only focusing on realizing a single functionality.

traffic and forwarded to the unseen classifier (customized model) at stage 4; otherwise, the sample is considered seen malicious traffic and forwarded to the seen classifier at stage 3. Given the discriminator has detected d seen malicious traffic samples $X_s = \{x_1^s, \dots, x_d^s\}$ and $m - d$ unseen malicious traffic samples $X_u = \{x_1^u, \dots, x_{m-d}^u\}$, the seen classifier $f_{seen} : X_s \rightarrow Y_s$ at stage 3 and unseen classifier $f_{unseen} : X_u \rightarrow Y_u$ at stage 4 output probability distribution and make predictions for each detected malicious traffic samples, where $Y_s = \{y_1, y_2, \dots, y_S\}$ and $Y_u = \{y_1, y_2, \dots, y_U\}$ denote the label space of seen and unseen classes. The unseen classifier is trained on the training set containing seen malicious traffic samples $X_t = \{x_1^s, \dots, x_t^s\}$ and auxiliary semantic information corresponding to the categories $A = \{a_1^s, \dots, a_e^s, a_1^u, \dots, a_f^u\}$ due to the lack of labeled training samples of unseen classes. The critical acronyms are summarized in Table II.

B. Discrimination

An interesting phenomenon we observe is that malicious traffic of the same category exhibit similar features, with their distribution in feature space being more concentrated, forming one or more continuous clusters. In contrast, malicious traffic of different categories tends to occupy distinct regions in the feature space. The nearest neighborhoods of seen malicious traffic samples typically contain more seen malicious traffic, which provides insights for discriminating seen from unseen

TABLE II
ACRONYMS SUMMARY

Acronyms	Definitions
HNIDS	Hybrid Network Intrusion Detection System
MADF	Misuse plus Anomaly Detector Framework
SG-ZSL	Semantic-Guided Zero-Shot Learning
NIDS	Network Intrusion Detection System
AUF	A Unified Framework
KNN	K-Nearest Neighborhood
ZSL	Zero-Shot Learning
OOD	Out-of-Distribution
HIoT	Bovenzi et al. [19]
DNN	Sun et al. [29]
CE-GZSL	Han et al. [30]
CO-GZSL	Li et al. [31]
CD-GZSL	Fan et al. [32]

malicious traffic. Given the malicious traffic samples detected in the first stage $X_a = \{x_1^s, \dots, x_d^s, x_{d+1}^u, \dots, x_m^u\}$, and the training set containing seen malicious traffic samples $X_t = \{x_1^s, \dots, x_t^s\}$, the union of these two sets is $X_{all} = \{x_1^s, \dots, x_d^s, x_{d+1}^u, \dots, x_m^u, x_1^s, \dots, x_t^s\}$. For seen malicious traffic $X_s = \{x_1^s, \dots, x_d^s\}$, their nearest neighbors in X_{all} are more likely to belong to the training set X_t . Conversely, for unseen malicious traffic $X_u = \{x_1^u, \dots, x_{m-d}^u\}$, their nearest neighbors in X_{all} mainly belong to the test set X_a .

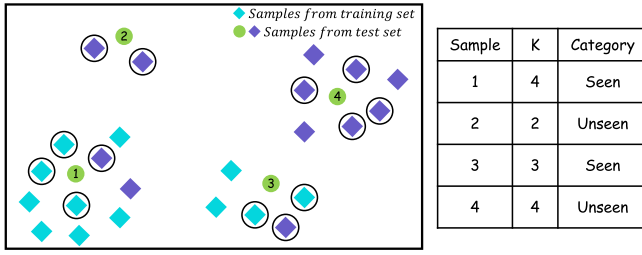


Fig. 3. Proposed adaptive KNN-based approach for discrimination. Based on the local density of each sample, we first compute the adaptive k value for each sample and determine whether it belongs to seen or unseen malicious traffic, depending on the number of samples belonging to the training set in the k -nearest neighbors.

Inspired by [29], we can discriminate between seen and unseen malicious traffic by calculating the nearest neighbors to the training set X_t . As a result, for each sample x_i from $X_a = \{x_1^s, \dots, x_d^s, x_{d+1}^u, \dots, x_m^u\}$, the first step is to calculate the Euclidean distance $d(x_i, x_j)$ between x_i and $X_{all} = \{x_1^s, \dots, x_d^s, x_{d+1}^u, \dots, x_m^u, x_1^s, \dots, x_t^s\}$:

$$d(x_i, x_j) = \|x_i - x_j\|_2, \quad (1)$$

Then, we sort the distances $d(x_i, x_j)$ in ascending order and select the top k nearest neighbors $NN_k(x_i) = \{x_{i_1}, x_{i_2}, \dots, x_{i_k}\}$. The OOD score for each sample $\text{score}(x_i)$ depends on the number of neighbors from seen categories within the fixed k nearest neighbors, which is calculated as follows:

$$\text{score}(x_i) = \sum_{\substack{x_{ij} \in NN_k(x_i) \\ j=1}}^k \mathbb{I}(x_{ij} \notin X_t) \quad (2)$$

where $\mathbb{I}(x_{ij} \notin X_t)$ denotes the indicator function that returns 0 if the neighbor x_{ij} is from the training set (i.e., $x_{ij} \in X_t$), and 1 otherwise.

An inevitable issue is that the fixed k value does not fit samples from all classes. For example, in the CIC-IDS2017 dataset, the Heartbleed category contains only nine samples, while the PortScan category contains 100,000 samples. Suppose k is fixed at a value larger than nine. In that case, the k -nearest neighbors for samples in the Heartbleed category will include many samples from other categories, which would interfere with the discrimination. Therefore, it is necessary to calculate an adaptive k value for each sample. As illustrated in Figure 3, given that samples from the majority classes tend to be higher in local density than samples from the minority classes, we calculate mean distances and determine the k value for each sample. Based on the distance calculated as Equation (1), we sort the distances $d(x_i, x_j)$ in ascending order and select the top $\hat{k}$ nearest neighbors $NN_{\hat{k}}(x_i) = \{x_{i_1}, x_{i_2}, \dots, x_{i_{\hat{k}}}\}$. $\hat{k}$ is a number larger than k for acquiring more accurate local density. Then we calculate mean distance $\bar{d}(x_i)$ of $\hat{k}$ nearest neighbors as follows:

$$\bar{d}(x_i) = \frac{1}{\hat{k}} \sum_{j=1}^{\hat{k}} d(x_i, x_{i_j}) \quad (3)$$

As mentioned above, samples from majority classes tend to have higher local density and require a higher k value. In contrast, samples from minority classes are prone to have lower local density and require a smaller k value. Thus, we normalize and invert the distance as follows:

$$\tilde{d}(x_i) = 1 - \frac{\bar{d}(x_i) - d_{\min}}{d_{\max} - d_{\min}}, \quad (4)$$

where $d_{\min}$ and $d_{\max}$ denote the minimum and maximum mean distances across all samples from test set X_a , respectively. Finally, the adaptive k value for each sample is determined as:

$$k_i = k_{\min} + \tilde{d}(x_i) \times (k_{\max} - k_{\min}), \quad (5)$$

where $k_{\min}$ and $k_{\max}$ denote the minimum and maximum values of k as specified in hyperparameters. Given the set of adaptive k -nearest neighbors $NN_{k_i}(x_i) = \{x_{i_1}, x_{i_2}, \dots, x_{i_{k_i}}\}$ for sample x_i , the OOD score is calculated as follows:

$$\text{score}(x_i) = \frac{k_i}{\sum_{\substack{x_{ij} \in NN_{k_i}(x_i) \\ j=1}}^{k_i} \mathbb{I}(x_{ij} \in X_t) + 1}, \quad (6)$$

where $\mathbb{I}(x_{ij} \in X_t)$ denotes the indicator function that returns 1 if the neighbor x_{ij} is from the training set (i.e., $x_{ij} \in X_t$), and 0 otherwise. Since unseen malicious traffic tends to have higher OOD scores, we can discriminate unseen malicious traffic by setting a threshold τ , where malicious traffic exceeding this threshold is recognized as unseen malicious traffic.

C. Unseen Malicious Traffic Classification

1) *Semantic Information Extraction*: As mentioned above, classifying unseen malicious traffic without labeled samples requires incorporating auxiliary semantic information for unseen classes. Therefore, extracting task-relevant and high-quality semantic information is the first crucial step. There are many ways to obtain auxiliary semantic information, such as expert knowledge and network information. We achieve this goal through SecureBERT [35], a domain-specific language model for cybersecurity. Specifically, we design prompts corresponding to categories, such as “A malicious traffic of PortScan.” and forward them into the SecureBERT. SecureBERT then produces a 768-dimensional word embedding representing the semantic information of the category. These embeddings serve as the auxiliary information for zero-shot classification.

2) *Feature Generation*: After obtaining auxiliary semantic information, the next step is to build a “bridge” between it and the features. As shown in Figure 4, we deploy Wasserstein GAN as the backbone architecture of the model. Specifically, Wasserstein GAN comprises a generator G and a discriminator D . The generator G takes Gaussian noise $\varepsilon \sim \mathcal{N}(0, 1)$ and semantic information a as input and generates fake features $\tilde{x}$. Real feature x and fake feature $\tilde{x}$ are concatenated with semantic information a and then fed into the discriminator D for adversarial training. Given semantic information a for unseen classes and noise z , we can generate features corresponding to the categories of malicious traffic with the trained

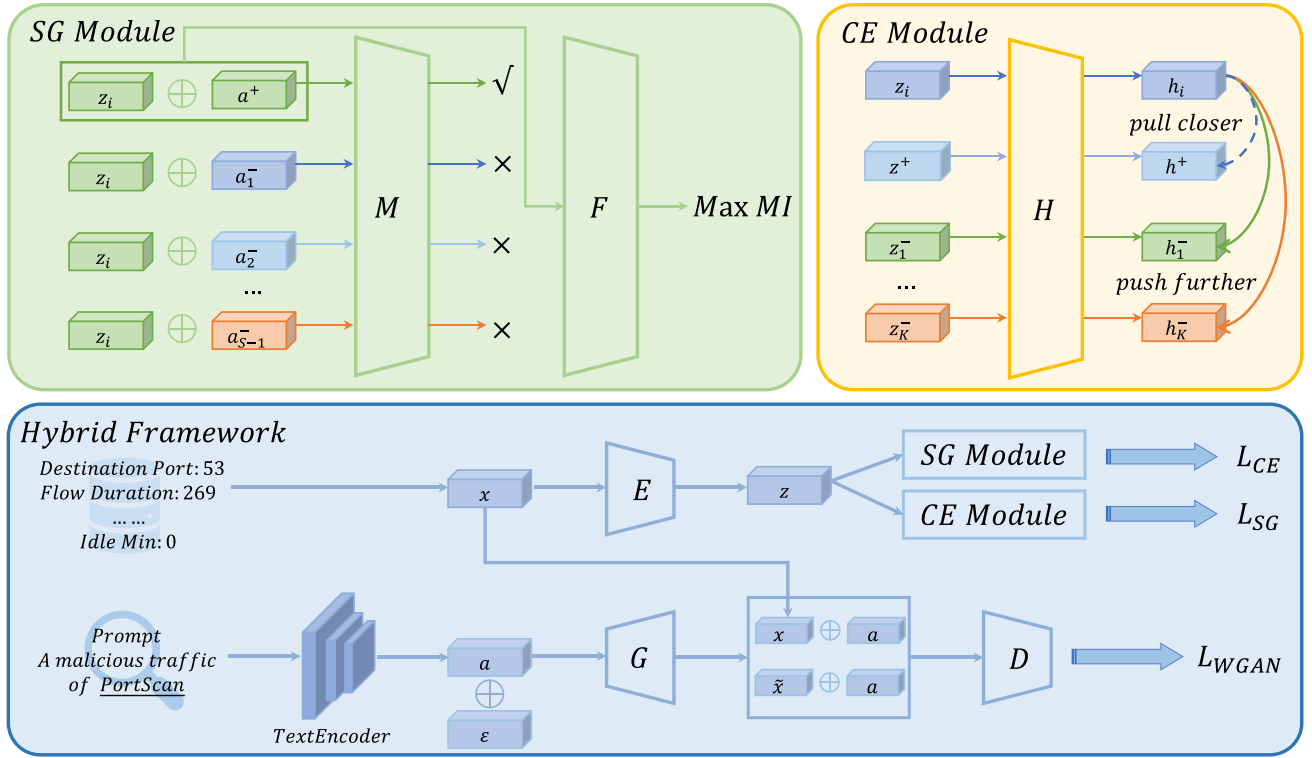


Fig. 4. Proposed SG-ZSL approach. The generator generates features that match the given semantic information; the semantic guidance module extracts embeddings with semantic information; the contrastive embedding module is responsible for enhancing the discriminative of embeddings.

generator. The generator and discriminator are optimized by the following objective:

$$\mathcal{L}_{WGAN} = \mathbb{E}[D(x, a)] - \mathbb{E}[D(\tilde{x}, a)] - \lambda \mathbb{E}[\|\nabla_{\tilde{x}} D(\tilde{x}, a)\|_2 - 1]^2, \quad (7)$$

where $\tilde{x} = G(\varepsilon, a)$ denotes the synthesized fake features, and the last term denotes the gradient penalty, where λ is the penalty coefficient.

3) *Semantic-Guided Module*: To further establish the correlation between feature and semantic information, we train the encoder E with the semantic-guided module, ensuring that the embedding $z = E(x)$ of the feature x contains as much semantic information as possible. On the one hand, we encourage the encoder E to generate embeddings matching the corresponding class semantics by training a matching network $M(z, a)$. The matching network accepts one positive pair of embedding and semantic information and $K - 1$ negative pairs and measures their correlation. If they belong to the same class, they are considered a match; otherwise, they are not. Concretely, we optimize the encoder E and the matching network M with the following constraint:

$$\mathcal{L}_{mat} = \mathbb{E}_{z_i, a^+} \left[-\log \frac{\exp(M(z_i, a^+)/\tau_s)}{\sum_{k=1}^K \exp(F(z_i, a_k)/\tau_s)} \right], \quad (8)$$

where a^+ denotes the semantic information corresponding to the same class as z_i , K denotes the number of seen malicious traffic categories, a_K denotes the semantic information different from the class of z_i , and τ_s denotes the temperature parameter.

On the other hand, we train a mutual information estimator $F(z, a)$ to guide the encoder E to learn embeddings related to semantic information. Compared to directly aligning the distributions of features and semantic information, maximizing mutual information helps preserve the features' diversity. We follow the approach proposed in [36] to estimate the mutual information between semantic information and features:

$$I(z_i, a_i) = \mathbb{E} \left[\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(F(z_i, a_i))}{\frac{1}{N} \sum_{j=1}^N \exp(F(z_i, a_j))} \right]. \quad (9)$$

Finally, we define the loss function of the semantic-guided module as follows:

$$\mathcal{L}_{sg} = \mathcal{L}_{mat} - \beta I(z_i, a_i), \quad (10)$$

where β denotes the weight for maximizing mutual information.

4) *Contrastive Embedding Module*: Learning discriminative features is also crucial since the ultimate goal is to train a classifier. We achieve this goal through the contrastive embedding module, which enhances intra-class compactness and inter-class separability in the embedding space. Specifically, for the original features x , we randomly sample a positive sample x^+ from the training batch and K negative samples $\{x_1^-, \dots, x_K^-\}$. Through the encoder E and non-linear projection head H , we aim to minimize the distance between $h_i = H(E(x))$ and $h_i^+ = H(E(x^+))$, while maximizing the distance between h_i and $\{h_1^-, \dots, h_K^-\}$. Specifically, we optimize the following supervised contrastive learning loss:

$$\mathcal{L}_{ce} = \mathbb{E}_{z_i, a^+} \left[-\log \frac{\exp(h_i^\top h^+/\tau_e)}{\exp(h_i^\top h^+/\tau_e) + \sum_{k=1}^K \exp(h_i^\top h_k^-/\tau_e)} \right], \quad (11)$$

where K denotes the number of negative samples, and τ_e denotes the temperature parameter.

5) *Overall Objective*: Combining the above objective functions, we formulate the total objective function of our hybrid framework as follows:

$$\mathcal{L}_{total} = \mathcal{L}_{WGAN}(G, D) + \mathcal{L}_{sg}(E, M, F) + \gamma \mathcal{L}_{ce}(E, H), \quad (12)$$

where $\mathcal{L}_{WGAN}(G, D)$ denotes the optimization of parameters for generator G and discriminator D , $\mathcal{L}_{sg}(E, M, F)$ denotes the optimization of parameters for encoder E , matching network M , and mutual information estimator F , γ denotes the weight of contrastive embedding loss, and $\mathcal{L}_{ce}(E, H)$ denotes the optimization of parameters for encoder E and non-linear projection head H .

6) *Classification*: We first need to collect auxiliary semantic information of unseen malicious traffic to classify unseen malicious traffic. As mentioned above, we feed the prompt containing the unseen class category into SecureBert and obtain the corresponding embedding as auxiliary semantic information. Next, we utilize the semantic information of unseen classes $a_1, \dots, a_U$ and the generator G to synthesize a set of features for each unseen classes $\tilde{x}_1^u, \dots, \tilde{x}_m^u$, where $\tilde{x}_i^u = G(\varepsilon, a_u)$. The embeddings $\tilde{z}_1^u, \dots, \tilde{z}_m^u$ can be obtained through the encoder E , where $\tilde{z}_i^u = E(\tilde{x}_i^u)$. We can train a classifier for unseen malicious traffic classification based on these discriminative embeddings containing sufficient semantic information and corresponding labels. The performance of the classifier is evaluated based on the prediction accuracy.

IV. EXPERIMENTAL SETUP

A. Dataset

We employ the BoT-IoT dataset [37] adopted by Bovenzi et al. [19] (abbreviated as HIoT) and the CIC-IDS2017 dataset [38] adopted by Verkerken et al. [20] to evaluate the effectiveness of the proposed AUF framework. The CIC-IDS2017 dataset includes raw PCAP traffic files and 78-dimensional feature files extracted using CICFlowMeter. As shown in Table III, the CIC-IDS2017 dataset encompasses 14 types of malicious traffic, including DoS, Portscan, Infiltration, and Bot attacks. The number of samples in different categories varies greatly. For example, the majority classes, such as Benign and DoS Hulk, contain 2,095,057 and 172,846 samples, respectively. In contrast, minority classes like Infiltration and Heartbleed have only 36 and 11 samples, respectively, highlighting that the network traffic is highly class-imbalanced.

The BoT-IoT dataset consists of raw PCAP traffic files and feature files containing the 10 selected features. As shown in Table III, the BoT-IoT dataset includes four categories of malicious traffic: DDoS, DoS, Reconnaissance, and Theft. Similar to CIC-IDS2017, the network traffic exhibits an imbalanced class distribution. The DDoS category, with the highest number of samples, comprises 1,926,624 instances, whereas the Theft category, with the fewest samples, includes only 79 instances. Notably, unlike CIC-IDS2017, the proportion of malicious traffic in BoT-IoT is significantly higher, with only

TABLE III
DETAILS OF CIC-IDS2017 AND BoT-IoT DATASET

Class	Abbreviation	Samples	Proportion
Benign	Benign	2095057	83.111%
DoS Hulk	Hulk	172846	6.857%
DDoS	DDoS	128014	5.078%
PortScan	PortScan	90694	3.598%
DoS GoldenEye	GoldenEye	10286	0.408%
FTP-Patator	FTP	5931	0.235%
DoS Slowloris	Slowloris	5385	0.214%
DoS Slowhttptest	Slowhttptest	5228	0.207%
SSH-Patator	SSH	3219	0.128%
Bot	Bot	1948	0.077%
Web Attack Brute Force	Brute Force	1470	0.058%
Web Attack XSS	XSS	652	0.026%
Infiltration	Infiltration	36	0.001%
Web Attack Sql Injection	Sql Injection	21	0.001%
Heartbleed	Heartbleed	11	0.000%
DDoS	DDoS	1926624	52.518%
DoS	DoS	1650260	44.984%
Reconnaissance	Scan	91082	2.483%
Normal	Normal	477	0.013%
Theft	Theft	79	0.002%

477 normal traffic samples, accounting for a mere 0.013% of the total dataset. We utilize all available samples for dataset splits without excluding any instances to maintain the class-imbalanced characteristics. We use the labeled CSV feature files provided by the CIC-IDS2017 and BoT-IoT datasets for dataset split. For the CIC-IDS2017 dataset, we adopt the same preprocessing strategy as [39] and retain 70-dimensional features. For the BoT-IoT dataset, we utilize the selected 10-dimensional features provided by the dataset. It is worth noting that the proposed AUF framework does not restrict the form of features processed by different models. Models based on flow-based statistics or pre-processed features as input are compatible with the AUF framework.

Figure 5 illustrates the dataset split on the CIC-IDS2017 dataset for evaluating different models. For evaluating the detector, we randomly designate eleven classes as seen malicious classes, while the remaining three serve as unseen malicious classes. The training set consists of samples from eight seen malicious classes and the benign class, whereas the validation set includes samples from all eleven seen malicious classes and the benign class. The test set used for detection contains samples from all fourteen malicious classes and the benign class. The dataset split follows a similar approach for evaluating the discriminator and the customized unseen malicious traffic classifier, with the only difference being the exclusion of the benign class. The training, validation, and test sets consist solely of samples from the eleven seen malicious classes for evaluating seen malicious traffic classifiers. The partitioning of seen and unseen classes results in 364 possible permutations, from which we randomly select 98 as the final dataset splits. The BoT-IoT dataset split utilizes the same strategy as the CIC-IDS2017 dataset, with the only difference being the number of classes in the training, validation, and test sets. Specifically, we randomly select two as seen malicious classes and the remaining two as unseen ones. Since this division results in six possible permutations, we retain all

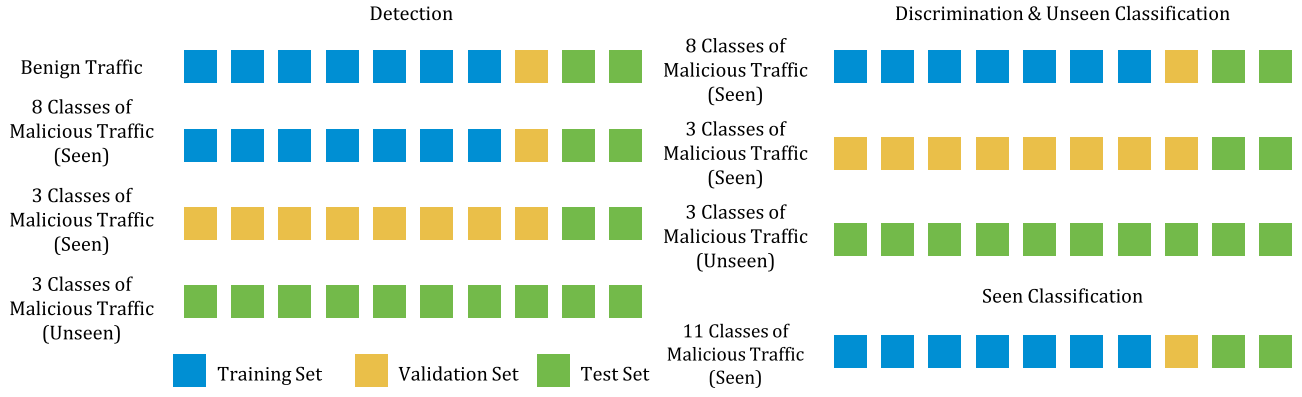


Fig. 5. Proposed dataset splits on CIC-IDS2017 dataset. Blue, yellow, and green colors indicate the training set, validation set, and test set samples, respectively.

of them as the final dataset splits. This partitioning strategy accounts for the presence of seen and unseen malicious traffic in real-world network environments and the challenges posed by class-imbalanced malicious traffic, ensuring a robust testbed for evaluating the proposed methods and framework.

B. Implement Details

We employ XGBoost from [40] using the default parameters for the detector, as XGBoost has been proven to perform well in anomaly detection and intrusion detection tasks [40], [41], [42], [43], [44], [45], [46]. We set the values of k in equation (2), $\hat{k}$ in equation (3), $k_{\max}$, and $k_{\min}$ in equation (5) to 20, 200, 20, and 3, respectively, for the discrimination of the CIC-IDS2017 dataset split. Given that the BoT-IoT dataset split contains a larger sample size, we increase $k_{\max}$ in equation (5) to 40. We utilize the widely used XGBoost with its default parameters for the seen classifier. We implement the feature extractor using a multilayer perceptron with Rectified Linear Unit (ReLU) activation and 128 hidden units for the SG-ZSL approach. The overall objective function is optimized using the Adam solver with a batch size of 2048 and a learning rate of $1e^{-4}$. The size of the hidden units for the generator G , discriminator D , and comparator network F are fixed at 128, 128, and 70, respectively. The weights of the contrastive embedding loss γ in equation (12) and the mutual information maximization β in equation (10) are set to 0.01. After the generator G and feature extractor E converge, we generate 10,000 samples for each class and train an XGBoost classifier as the unseen classifier. Since the BoT-IoT dataset contains only four malicious traffic categories, we only compare different unseen classifiers on the CIC-IDS2017 dataset. All data splits utilize the same hyperparameter settings. The reported data are average results across all dataset splits. The figures without specified dataset split adopt the same dataset split: for the CIC-IDS2017 dataset, we adopt the split with the BoT, DDoS, and Slowloris classes specified as the unseen malicious traffic classes. For the BoT-IoT dataset, we adopt the split with the DDoS and Scan classes specified as the unseen malicious traffic classes.

C. Evaluation Metrics

We evaluate the performance of different models using different metrics. Specifically, we utilize Area Under the Receiver Operating Characteristic Curve (AUC-ROC), Area Under the Precision-Recall Curve (AUC-PR), Micro Recall, and Macro Recall to measure the performance of the detector and the discriminator, and the Micro Recall and Macro Recall to evaluate the performance of both seen and unseen classifiers. All methods adopt a consistent thresholding strategy to determine binary decisions based on detection scores. Specifically, we define a fixed True Positive Rate (TPR) as the thresholding criterion: $\text{TPR} = 0.99$ for two datasets. These thresholds are applied uniformly to the output scores from the detector, discrimination stage, and extension stage of Verkerken et al. [20] to ensure fair comparison. In addition to assessing the performance of each model individually, we cascade the detector, discriminator, classifier, and customized model to evaluate the overall classification performance. We treat wrongly detected benign traffic as malicious traffic and assign malicious labels to the wrongly detected benign traffic with the discriminator and classifier. At the same time, we assign wrongly detected malicious traffic benign labels. For wrongly discriminated malicious traffic, we assign labels from unseen malicious classes to wrongly discriminated seen malicious traffic and labels from seen malicious classes to wrongly discriminated unseen malicious traffic.

To demonstrate the superiority of our proposed adaptive KNN-based approach, we introduce HIoT [19] and DNN (ICML 2022) [29] for comparison. HIoT identifies traffic as unseen malicious traffic if the maximum value of the predicted probability is less than the threshold. DNN calculates the k -nearest neighbors for each test sample and considers the distance to the k -th nearest neighbor as the anomaly score. We select HIoT and DNN as representative baselines based on two criteria. On the one hand, HIoT shares structural similarities with the proposed AUF framework for hybrid network intrusion detection. On the other hand, our proposed adaptive KNN-based method is directly inspired by the out-of-distribution detection approach in DNN. We do not include Verkerken et al.'s method in the comparison because it takes

the same method for recognizing unseen malicious traffic as HIoT. For fair comparison with the DNN method, we calculate the K nearest neighbors of each test set sample in the training set and select the distance between the sample and the K th nearest neighbor as the OOD score. The value of parameter K is the same as $k_{\max}$ in equation (5): 20 for the CIC-IDS2017 dataset and 40 for the BoT-IoT dataset. We directly use the raw features rather than training additional feature extractors.

Regarding unseen classification, we compare the proposed SG-ZSL with three SOTA contrastive embedding-based ZSL methods: CE-GZSL [30], CO-GZSL [31], and CD-GZSL [32]. Since these approaches share the same backbone and training strategy proposed by CE-GZSL, we implement all methods using the official codes, ensuring that all hyperparameters except for method-specific are the same for fair comparison.

We also compare the hybrid performance of the AUF framework, HIoT, and Verkerken et al.'s method. For fair comparison, these three methods employ XGBoost as the detector and classifier. We select XGBoost since it yields better performance than unsupervised models and is commonly used in intrusion detection studies.

V. EVALUATION

A. Detection Results

1) *Performance*: Table IV shows the performance of XGBoost model as the detector and the seen classifier. As one of the best anomaly detectors, XGBoost achieves high performance across all evaluation metrics. For the CIC-IDS2017 dataset, XGBoost achieves an AUC-ROC of 0.9714 and an AUC-PR of 0.9525. While on the BoT-IoT dataset, XGBoost performs better with an AUC-ROC of 0.9988 and an AUC-PR of 1.0, indicating its ability to differentiate between benign and malicious samples. A notable observation is the gap between Macro Recall and Micro Recall in the two datasets. XGBoost achieves Micro Recall of 0.8671 and 0.9999 on CIC-IDS2017 and BoT-IoT datasets, respectively. However, the Macro Recall of 0.9455 and 0.9162 suggest a disparity in detection performance across different categories. This discrepancy mainly results from class imbalance, where certain classes of malicious traffic have fewer samples in the dataset, leading to suboptimal Macro Recall for minority classes. This discrepancy also highlights a common challenge in anomaly detection: ensuring that models generalize well to all classes, especially to minority classes.

To further investigate the decline in Macro Recall, we calculate the class-wise recall of XGBoost algorithm across 98 dataset splits in Table VI. The class imbalance problem is obviously one of the important reasons for the difference between Macro Recall and Micro Recall. Classes such as DDoS, DoS Hulk, and DoS GoldenEye achieve an average recall of 0.99, demonstrating that the detector can accurately identify most malicious traffic. However, the recall for some minority classes is relatively low, especially when these classes are unseen during training. Figure 6 illustrates the class-wise recall of three dataset splits on the CIC-IDS2017 dataset. For instance, when Heartbleed, SSH, and XSS are unseen classes, the recall for SSH and Heartbleed drops to zero. Similarly,

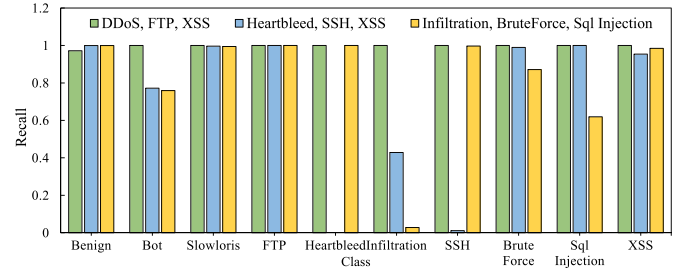


Fig. 6. Class-wise recall of three dataset splits for detection.

when Infiltration, BruteForce, and SQL Injection are unseen classes, the recall for categories such as Bot, Infiltration, and SQL Injection is unsatisfying. The decline in minority classes suggests that the detector's performance depends on the number of available training samples and classes. In addition to the class imbalance problem, the setting of thresholds also affects Micro Recall and Macro Recall. For the CIC-IDS2017 dataset, while 99% of malicious traffic is detected, a large amount of benign traffic is also misdetected as malicious traffic. The recall of benign traffic is 0.8163, which is lower than the recall of most malicious traffic categories. Since benign traffic accounts for the majority of traffic, Micro Recall is lower than Macro Recall.

Table IV also presents the speed differences between the AUF and MADF in distinguishing benign and malicious traffic. Both frameworks employ XGBoost as the detector and classifier for fair comparison. On the CIC-IDS2017 and BoT-IoT datasets, the AUF completes malicious traffic detection in 0.1522s and 0.0957s, respectively. In contrast, on the CIC-IDS2017 dataset, MADF takes 0.5267s to complete malicious traffic detection, consuming more than three times the inference time of the AUF. On the BoT-IoT dataset, MADF requires 0.1809s, which is still nearly twice the time required by the AUF. The primary reason for this discrepancy is that the classifier significantly increases the overall inference time. The detector is deployed in the first stage for the AUF, allowing it to differentiate between benign and malicious traffic solely through the detector. The MADF requires the majority of benign traffic to be processed by both the misuse detector (i.e., classifier) and the anomaly detector (i.e., detector). Although the classifier detects some malicious traffic in the first stage, thereby reducing the detector's inference time, the time cost remains high. The design of the AUF framework enhances malicious traffic detection speed. It is expected to achieve higher accuracy and faster inference when incorporating more advanced detectors, such as [47], [48].

2) *Impact on Unseen Malicious Traffic*: Class-imbalanced traffic and unseen malicious traffic are challenges faced by all HNIDSs. Class-imbalanced traffic affects the recall of minority classes, while unseen malicious traffic impacts the detection performance of both the majority class and the minority class. Taking the CIC-IDS2017 dataset as an example, the Bot class achieves an average recall of only 0.8848 with 1948 samples, lower than the Brute Force (0.9954) and XSS (0.9947) classes with only 1470 and 652 samples, respectively. To further investigate the impact of unseen malicious traffic, we set two experimental scenarios: (1) all malicious traffic classes are

TABLE IV
PERFORMANCE OF THE DETECTOR AND SEEN CLASSIFIER

Detection						
Dataset	AUC-ROC	AUC-PR	Micro Recall	Macro Recall	Time (AUF)	Time (MADF)
CIC-IDS2017	0.9715	0.9525	0.8671	0.9455	0.1522s	0.1321s
Bot-IoT	0.9988	1.0000	0.9901	0.9162	0.0957s	0.0518s
Classification						
Dataset	AUC-ROC	AUC-PR	Micro Recall	Macro Recall	Time (AUF)	Time (MADF)
CIC-IDS2017	-	-	0.9984	0.9187	-	0.3946s
Bot-IoT	-	-	0.9999	0.9843	-	0.1291s

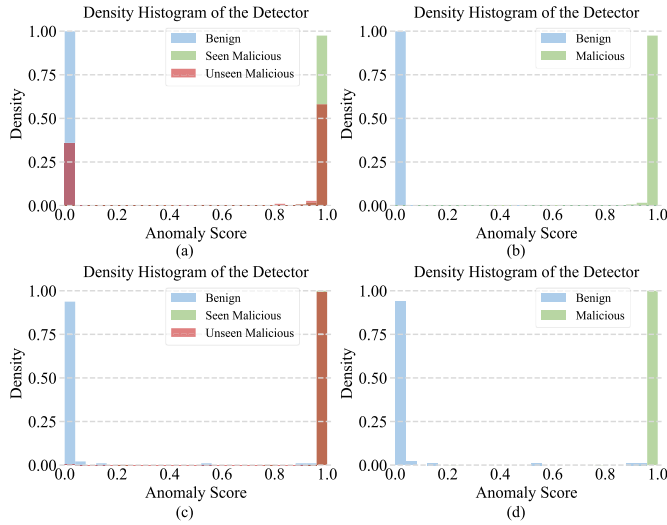


Fig. 7. Density histogram of the detector on two datasets. (a) Density histogram on the CIC-IDS2017 dataset with the test set includes both seen and unseen malicious traffic. (b) Density histogram on the CIC-IDS2017 dataset with the test set contains only seen malicious traffic. (c) Density histogram on the Bot-IoT dataset with the test set includes both seen and unseen malicious traffic. (d) Density histogram on the Bot-IoT dataset with the test set contains only seen malicious traffic.

available during the training phase, and the test set contains only seen malicious traffic, and (2) some classes of malicious traffic are unavailable during the training phase, the test set includes both seen and unseen malicious traffic, consistent with our proposed dataset split.

Figure 7 illustrates XGBoost's performance under two scenarios. For the CIC-IDS2017 dataset, seen malicious traffic generally exhibits high anomaly scores, while benign traffic maintains low anomaly scores. This difference explains why XGBoost performs well in anomaly detection - choosing a threshold between 0.1 and 0.9 enables accurate detection. However, the anomaly scores of unseen malicious traffic differ from those of seen classes, with only approximately 60% of unseen malicious traffic having high anomaly scores. Nearly 40% of the unseen malicious traffic is low in anomaly score, leading to confusion between benign and malicious traffic. For the Bot-IoT dataset, both seen and unseen malicious traffic exhibit high anomaly scores. Nevertheless, around 1% of unseen malicious traffic has anomaly scores similar to benign traffic. This discrepancy increases the likelihood of

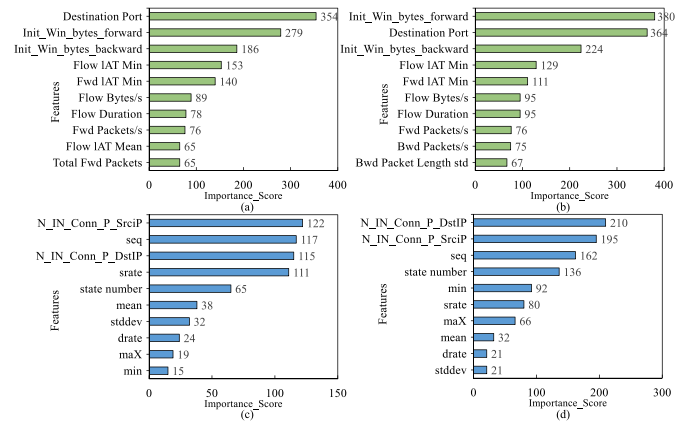


Fig. 8. Feature importance score of the detector on two datasets. (a) Feature importance score on the CIC-IDS2017 dataset with the test set includes both seen and unseen malicious traffic. (b) Feature importance score on the CIC-IDS2017 dataset with the test set contains only seen malicious traffic. (c) Feature importance score on the Bot-IoT dataset with the test set includes both seen and unseen malicious traffic. (d) Feature importance score on the Bot-IoT dataset with the test set contains only seen malicious traffic.

missed detection of unseen malicious traffic, highlighting the necessity of designing the detector with generalization for unseen malicious traffic.

Leveraging XGBoost's interpretability, we analyze feature importance variations to understand XGBoost's performance under different scenarios. Figure 8 shows the feature importance scores of XGBoost in both experimental settings. For the CIC-IDS2017 dataset, the eight most important features remain unchanged across scenarios. Notably, five features—Destination Port, Init_Win_bytes_forward, Init_Win_bytes_backward, Flow IAT Min, and Fwd IAT Min—consistently exhibit importance scores above 100, indicating their crucial role in malicious traffic detection.

However, the rankings and scores of individual features undergo noticeable shifts. Flow IAT Mean and Total Fwd Packets emerge as the ninth and tenth most important features, each with an importance score of 65 when the test set includes both seen and unseen malicious traffic. Conversely, in the scenario where only seen malicious traffic exists, Bwd Packets/s and Bwd Packet Length Std take the ninth and tenth positions with scores of 75 and 67, respectively. Additionally, Init_Win_bytes_forward increases in importance from 279 to 380, surpassing Destination Port as the most critical feature.

TABLE V
PERFORMANCE COMPARISON BETWEEN DIFFERENT DISCRIMINATORS

CIC-IDS2017				
Method	AUC-ROC	AUC-PR	Micro Recall	Macro Recall
HIoT [19]	0.9200	0.8191	0.8227	0.4857
DNN [29]	0.8239	0.7817	0.6003	0.3824
w/o ada (Ours)	0.9983	0.9946	0.9622	0.9081
Full (Ours)	0.9983	0.9943	0.9640	0.9128

BoT-IoT				
Method	AUC-ROC	AUC-PR	Micro Recall	Macro Recall
HIoT [19]	0.5926	0.7853	0.7467	0.4994
DNN [29]	0.8586	0.7754	0.7503	0.5648
w/o ada (Ours)	0.9995	0.9994	0.9933	0.9347
Full (Ours)	0.9995	0.9994	0.9926	0.9669

The overall ranking significantly changes for the BoT-IoT dataset, which consists of only ten features. In the test set comprising solely seen malicious traffic, the feature `N_IN_Conn_P_DstIP` surpasses `N_IN_Conn_P_SrcIP` and seq with an importance score of 210, emerging as the most significant feature. Meanwhile, the importance score of the feature `srate` drops from 111 to 80, shifting from the fourth most important feature to the sixth. These variations in feature importance explain XGBoost's suboptimal performance in detecting unseen malicious traffic. Since unseen malicious traffic is absent during training, XGBoost fails to emphasize features crucial for distinguishing unseen malicious traffic, such as `Init_Win_bytes_forward` and `N_IN_Conn_P_SrcIP`.

B. Discrimination Results

1) *Performance*: Table V compares four discriminators' performance on the CIC-IDS2017 and the BoT-IoT dataset. We observe that the adaptive KNN-based method achieves exceptional performance, with average AUC-ROC and AUC-PR exceeding 0.99, indicating that the adaptive KNN-based method effectively distinguishes most malicious traffic. Moreover, introducing the adaptive k strategy boosts the method's performance in Macro Recall, which improves the average Macro Recall by 0.004. The adaptive k strategy allows the method to dynamically adjust each sample's k value based on the local density, leading to a more balanced recall across all classes.

The proposed adaptive KNN-based method also significantly outperforms the other approaches in all metrics. On the CIC-IDS2017 dataset, the proposed method outperforms the baseline by at least 0.0783 in AUC-ROC, 0.1752 in AUC-PR, and 0.1413 in Micro Recall. On the BoT-IoT dataset, the proposed method achieves a minimum improvement of 0.1409 in AUC-ROC, 0.2141 in AUC-PR, and 0.236 in Micro Recall. Two primary factors contribute to the significant improvements. On the one hand, the adaptive KNN-based method leverages the class distribution in the training and test sets rather than relying solely on the training set for predictions. On the other hand, by utilizing the k -nearest neighbors' distances and class distribution, the adaptive KNN-based method makes more informed decisions than DNN, where the DNN only considers the k -nearest neighbors' distance and

HIoT only considers the output of the classifier. Two primary factors allow the adaptive KNN-based method to capture the class distribution disparities between seen and unseen classes, leading to more accurate discrimination.

Introducing the adaptive k strategy in the adaptive KNN-based discriminator improves the Macro Recall. However, it still lags behind Micro Recall by 0.0512 on the CIC-IDS2017 dataset and 0.0257 on the BoT-IoT dataset, respectively. This discrepancy indicates that the recall for certain minority classes is lower than for others. The class-wise recall in Table VI provides further insight into this issue. For the majority classes, such as Dos, DoS Hulk, DDoS, and PortScan, the recall is all above 0.96. For the minority classes, such as Scan, SSH-Patator and Bot, recall exceeds 0.96. The results explain why the adaptive KNN-based method performs well regarding AUC-ROC, AUC-PR, and Micro Recall. However, the recall drops for classes with fewer than 1500 samples. The average recall for Theft, Brute Force, XSS, Infiltration, Sql Injection, and Heartbleed are 0.8869, 0.8497, 0.7918, 0.8473, 0.7221, and 0.8692, respectively. The lack of sufficient samples significantly contributes to the decline in recall. The insufficient number of samples leads to a situation where the nearest neighbors of these classes' samples may include samples from other classes, thereby interfering with the discrimination. Even though the adaptive k strategy helps to some extent, it can only partially mitigate the impact of insufficient samples.

Further analysis reveals that the reasons for the decline in recall vary among different minority classes. A closer examination of different data splits and class-wise recall shows that if XSS and Brute Force are both split as seen malicious classes, their recall is nearly 1. However, the recall drops significantly if either XSS or Brute Force is split as an unseen malicious class. Since both classes belong to the Web Attack and share similar features, the discrimination becomes challenging when these two classes are split into different categories, indicating that the adaptive KNN-based method still has room for improvement in discriminating between classes with similar features. For classes of Theft, Sql Injection, Heartbleed, and Infiltration, their recall is closely related to their distribution in the feature space. The last row of Table VI shows the average distance between each class's samples and their adaptive k -nearest neighbors. Heartbleed and Infiltration have higher average distances than other classes, with values of 2.4790 and 1.4458, respectively. As shown in Equation (5), since the adaptive k value increases linearly as the average distance decreases, the adaptive k value for Infiltration samples will be significantly higher than that for Heartbleed. However, with only 36 samples, the higher k value for Infiltration results in nearest neighbors that include samples from other classes, affecting the final discriminating. On the other hand, the average distance for the Sql Injection class is only 0.0742. Its relatively high k value and the sparse 21 samples result in the recall that ranks lowest among the fourteen classes. Similar to the Sql Injection class, the number of samples in the Theft class is much smaller than that of other classes. Even if a lower adaptive k value is set, avoiding interference from samples of other classes is difficult.

TABLE VI
AVERAGE CLASS-WISE RECALL AND AVERAGE DISTANCE ON THE CIC-IDS2017 AND THE BoT-IoT DATASET

Class	Detection	Discrimination	Classification	Customized	Hybrid (Ours)	Hybrid (HIoT [19])	Hybrid (with Customized)	Hybrid ([20])	Average Distance
Benign	0.8163	-	-	-	0.8163	0.8163	0.8163	0.8163	-
Bot	0.8848	0.9719	0.9999	0.4782	0.8619	0.3924	0.8026	0.3876	0.0721
DDoS	0.9987	0.9649	1.0000	0.9461	0.9636	0.6431	0.9533	0.6418	0.0187
GoldenEye	0.9965	0.9668	0.9980	0.8426	0.9618	0.5421	0.9299	0.5421	0.0627
Hulk	0.9982	0.9651	0.9998	0.7270	0.9633	0.8075	0.9170	0.8062	0.0291
Slowhttptest	0.9913	0.9623	0.9952	0.8222	0.9505	0.4484	0.9179	0.4481	0.0512
Slowloris	0.9979	0.9715	0.9956	0.7435	0.9672	0.5331	0.9075	0.5330	0.0684
FTP	0.9806	0.9641	0.9998	0.7700	0.9446	0.6057	0.8981	0.6017	0.0035
Heartbleed	0.8033	0.8692	0.9156	0.9437	0.7041	0.0380	0.7041	0.0380	2.4790
Infiltration	0.7796	0.8473	0.8590	0.5764	0.6445	0.0946	0.6431	0.0917	1.4458
PortScan	0.9950	0.9650	0.9996	0.4833	0.9599	0.8866	0.8321	0.8838	0.0091
SSH	0.9766	0.9667	0.9999	0.7165	0.9437	0.6237	0.9018	0.6165	0.0077
Brute Force	0.9925	0.8497	0.8870	0.7398	0.7558	0.2648	0.7077	0.2645	0.0198
Sql Injection	0.9764	0.7221	0.7105	0.8918	0.5033	0.2026	0.4979	0.2021	0.0742
XSS	0.9942	0.7918	0.4975	0.8885	0.3976	0.1761	0.3869	0.1758	0.0209
Micro Recall	0.8163	0.9640	0.9984	0.7092	0.8601	0.8283	0.8242	0.8272	-
Macro Recall	0.8848	0.9128	0.9187	0.7525	0.8225	0.4717	0.7878	0.4700	-
Normal	0.9878	-	-	-	0.9878	0.9878	-	0.9878	-
DDoS	0.9987	0.9929	0.9999	-	0.9916	0.5119	-	0.5079	0.0147
DoS	0.9906	0.9930	0.9999	-	0.9838	0.4982	-	0.4935	0.0122
Scan	0.8819	0.9950	0.9999	-	0.8773	0.4431	-	0.4311	0.0080
Theft	0.7220	0.8869	0.9375	-	0.6635	0.2637	-	0.1540	0.1484
Micro Recall	0.9901	0.9926	0.9999	-	0.9828	0.7376	-	0.7310	-
Macro Recall	0.9162	0.9669	0.9843	-	0.9008	0.5409	-	0.5149	-

2) *Parameter Sensitivity*: Given that our method is based on the KNN algorithm, different values of k will affect the discrimination results. Therefore, we set various hyperparameters and investigate their impact on the adaptive KNN-based discriminator. As shown in Figure 9 (a) and Figure 10 (a), we first explore the changes in different metrics without applying the adaptive k strategy and set k to various values. The AUC-PR, AUC-ROC, and Micro Recall consistently remain close to 1, indicating that the model correctly discriminates most malicious traffic across different k values. Micro Recall shows a slight increase as k increases, though this increase is not significant enough to be visible in the graph. For the majority classes, such as Dos, DoS Hulk, DDoS, and PortScan, which constitute the majority of malicious traffic, setting a higher k value helps accurately discriminate samples of these classes and results in higher Micro Recall.

Unfortunately, as k increases, Macro Recall gradually decreases. When k is set to 20, 30, 40, and 50, Macro Recall decreases progressively from 0.9228 to 0.909, 0.902, and 0.899 on the CIC-IDS2017 dataset. On the BoT-IoT dataset, the macro recall increases by 0.001 when k is set to 30. However, as k rises, the macro recall still declines from 0.966 to 0.934. For some minority classes, such as Infiltration, SQL Injection, and Heartbleed, which have fewer than 50 samples each, setting a smaller k value is more conducive to accurately discriminating malicious traffic in these classes, thereby achieving higher Macro Recall. Our proposed adaptive KNN-based approach is actually based on this concept, assigning smaller k values to minority classes and larger k values to the majority classes, thereby achieving balanced and superior performance across all classes.

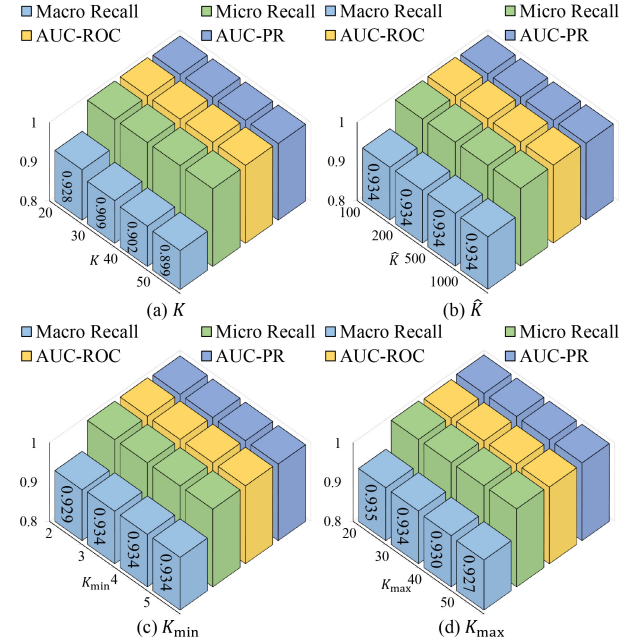


Fig. 9. Parameter sensitivity of the proposed adaptive KNN-based method on the CIC-IDS2017 dataset. (a) Model performance with different k . (b) Model performance with different $\hat{k}$. (c) Model performance with different $k_{\min}$. (d) Model performance with different $k_{\max}$.

Estimating local density is crucial to assign reasonable k values to different samples. We estimate density using the average distance of different samples to their nearest neighbors, with $\hat{k}$ being the key hyperparameter. As illustrated in Figure 9 (b) and Figure 10 (b), we fix the other hyperparameters and set $\hat{k}$ to 100, 200, 500, and 1000 to observe its

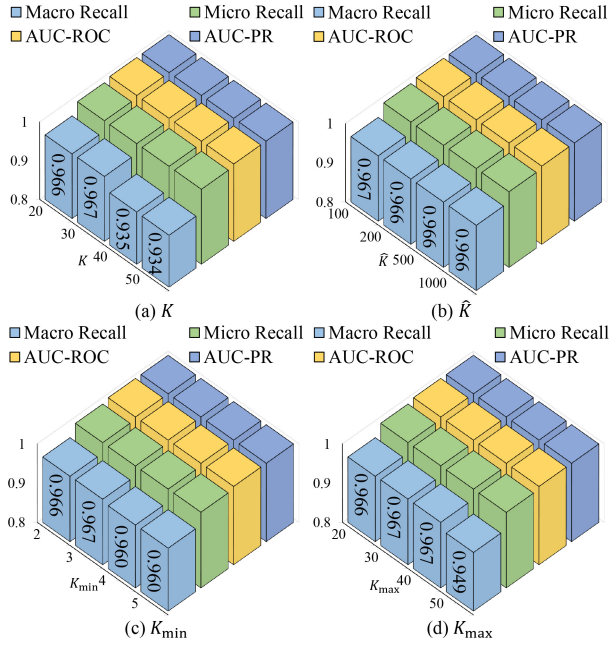


Fig. 10. Parameter sensitivity of the proposed adaptive KNN-based method on the BoT-IoT dataset. (a) Model performance with different k . (b) Model performance with different $\hat{k}$. (c) Model performance with different $k_{\min}$. (d) Model performance with different $k_{\max}$.

effect on the discrimination results. The results indicate that different values of $\hat{k}$ have little impact on the final results, which consistently remain around 0.934 and 0.966 on the CIC-IDS2017 and BoT-IoT dataset, respectively, suggesting that the adaptive KNN-based method is not sensitive to $\hat{k}$. Taking the CIC-IDS2017 dataset as an example, due to the lower local density of the Heartbleed and Infiltration classes, different $\hat{k}$ values enable smaller k values for samples of these classes. On the other hand, the SQL Injection class, due to its proximity to Brute Force and XSS samples, exhibits significantly higher local density compared to Heartbleed and Infiltration. Therefore, different $\hat{k}$ values fail to assign smaller k values to it, making the adaptive KNN-based model not sensitive to $\hat{k}$.

As shown in Figure 9 (c)(d) and Figure 10 (c)(d), the effect of setting different $k_{\max}$ and $k_{\min}$ values on the model is similar to that of setting different k values. When $k_{\min}$ is set to 3, the model performs best in Macro Recall, indicating that a smaller $k_{\min}$ helps the model assign smaller k values to minority classes, thereby improving performance on Macro Recall. When $k_{\max}$ is set to 20 on the CIC-IDS2017 dataset and 40 on the BoT-IoT dataset, the model achieves the best performance in terms of Macro Recall, mirroring the effect of increasing k value on the model's performance. As $k_{\max}$ increases, the model becomes more accurate in discriminating majority classes, but this simultaneously affects the discrimination of minority classes. Therefore, setting $k_{\max}$ to 20 on the CIC-IDS2017 dataset and 40 on the BoT-IoT dataset strikes a balance between Macro Recall and Micro Recall, achieving high recall for both majority and minority classes.

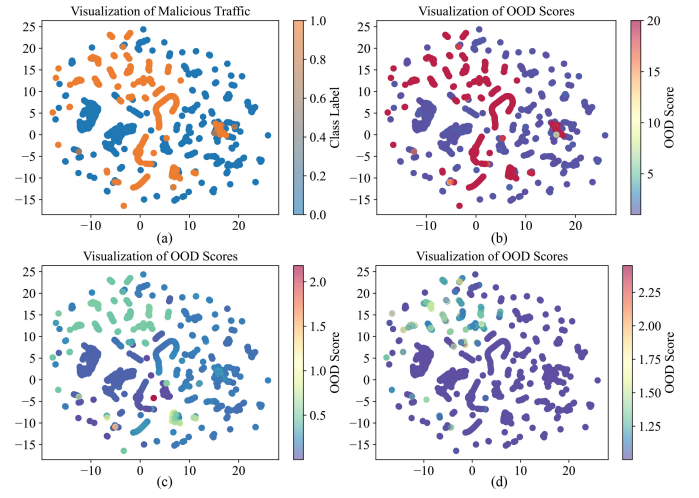


Fig. 11. The UMAP visualization of the malicious traffic features and the OOD scores (a) Malicious traffic features. (b) OOD scores of the proposed adaptive KNN-based method. (c) OOD scores of the KNN. (d) OOD scores of the HIoT.

3) *Visualization*: To intuitively demonstrate the superiority of the proposed method, we utilize UMAP for dimensionality reduction to visualize feature distributions and the OOD scores assigned by different discriminators. Figure 11 illustrates the performance of three discriminators on a specific split of the CIC-IDS2017 dataset. In Figure 11 (a), blue samples represent seen malicious traffic, while orange samples denote unseen malicious traffic. The feature space exhibits a structured distribution, where seen and unseen malicious traffic form distinct clusters. In the meantime, clusters of seen malicious traffic maintain a certain degree of separation between clusters of unseen malicious traffic. The distribution disparity enables our proposed adaptive KNN-based method to differentiate between seen and unseen malicious samples effectively. Figure 11 (b) shows that the OOD scores of seen and unseen malicious traffic differ significantly. Almost all seen malicious traffic receives low OOD scores, whereas unseen malicious traffic is assigned substantially higher scores. Even in the lower-right region where seen and unseen malicious traffic partially overlap, most samples retain OOD scores that align with ground truth.

In contrast, both the DNN method and the HIoT exhibit deviations between predicted OOD scores and the ground truth. Figure 11 (c) shows that the DNN method performs well in detecting the dispersed unseen malicious traffic in the upper-left region. However, the unseen malicious traffic is incorrectly assigned low OOD scores in the overlapping lower-right region. Since DNN determines OOD scores solely based on training set features, unseen malicious traffic in overlapping regions is influenced by malicious traffic of the training set, leading to incorrect OOD scores. The proposed adaptive KNN-based method overcomes this limitation- as many test samples also exist in overlapping regions, K-nearest neighbors of the unseen malicious traffic contain fewer samples from the training set, yielding more accurate OOD scores. HIoT predicts OOD scores based on classifier probability distributions, and it is difficult to explain the reason for decision-making based

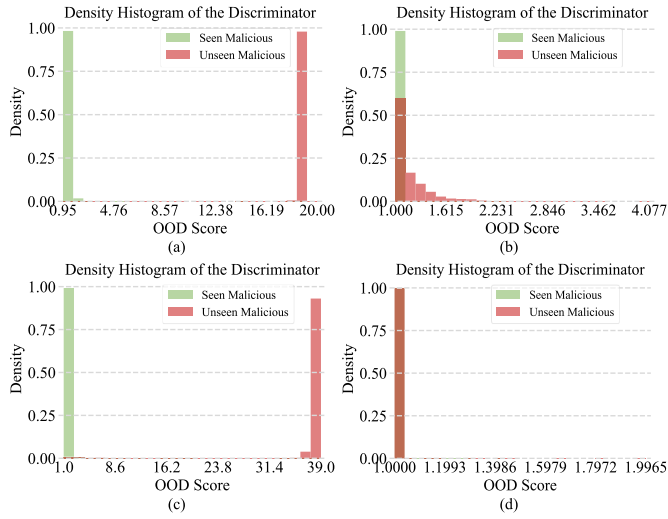


Fig. 12. Density histogram of the two discriminators on two datasets. (a) Density histogram of the proposed adaptive KNN-based method on the CIC-IDS2017 dataset. (b) Density histogram of the HIoT on the CIC-IDS2017 dataset. (c) Density histogram of the proposed adaptive KNN-based method on the BoT-IoT dataset. (d) Density histogram of the HIoT on the BoT-IoT dataset.

on feature distribution. As depicted in Figure 11 (d), only the upper-left region's unseen malicious traffic is assigned relatively high OOD scores. In contrast, most unseen malicious traffic receives scores similar to those of seen malicious traffic.

While UMAP visualization explains the effectiveness of the proposed method, it does not provide insights into selecting an optimal threshold. To address this, we visualize the density histograms of OOD scores to compare the differences between the adaptive KNN-based approach and HIoT. As shown in Figure 12 (a) and Figure 12(c), the proposed adaptive KNN-based method exhibits a clear decision boundary. Nearly all seen malicious traffic receives an OOD score close to 1, while most unseen malicious traffic is assigned significantly higher scores, around 20 for the CIC-IDS2017 dataset and 40 for the BoT-IoT dataset. Taking the CIC-IDS2017 dataset as an example, any threshold within the range of 1–20 can effectively separate seen from unseen malicious traffic.

In contrast, the performance of HIoT is suboptimal. As depicted in Figure 12 (b), approximately 60% of unseen malicious traffic in the CIC-IDS2017 dataset have OOD scores similar to seen malicious traffic, resulting in an indistinct separation. The performance further deteriorates on BoT-IoT, where, as shown in Figure 12 (d), OOD scores for both seen and unseen malicious traffic overlap in the narrow range of 0–0.08. The overlap leads to two issues: (1) selecting an appropriate threshold becomes challenging, and (2) even with an optimal threshold, distinguishing between seen and unseen malicious traffic remains difficult. The density histograms explain why HIoT achieves only 0.5926 AUC-ROC on the BoT-IoT dataset.

C. Classification

Regarding the classification of seen malicious traffic, XGBoost achieves an average Micro Recall of 0.9984 and

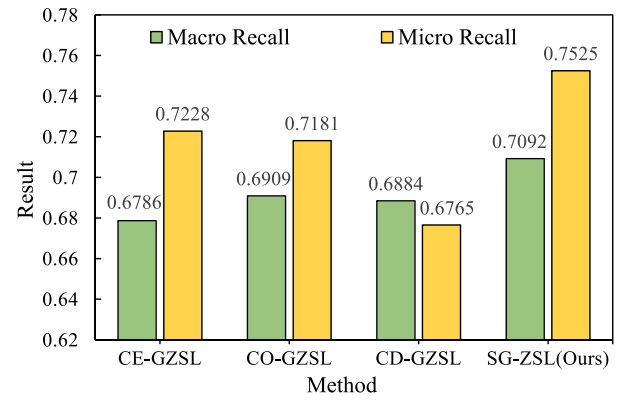


Fig. 13. Micro Recall and Macro Recall of four unseen classifiers.

0.9999 on the CIC-IDS2017 dataset and BoT-IoT dataset, respectively, reflecting XGBoost's ability to accurately classify the vast majority of traffic into the correct categories. Classifiers for classifying unseen malicious traffic also achieve satisfying recall despite lacking training samples, as illustrated in Figure 13. The four unseen classifiers obtain an average Micro Recall and Macro Recall exceeding 0.65 on the CIC-IDS2017 dataset, indicating the feasibility of applying zero-shot learning in the customized model. The classifiers trained with zero-shot learning require only auxiliary semantic information from administrators to classify most unseen malicious traffic correctly. Furthermore, the proposed SG-ZSL demonstrates superior performance compared to other SOTA methods. Specifically, SG-ZSL leads in both average Macro Recall and Micro Recall, with a 0.0297, 0.0397, and 0.0433 advantage in average Macro Recall and 0.0306, 0.0183, and 0.0208 in average Micro Recall over CE-GZSL, CO-GZSL, and CD-GZSL, respectively. The superiority highlights the effectiveness of the semantic-guided module in enhancing the relationship between traffic features and auxiliary semantic information. The module facilitates better alignment between the semantic and feature spaces, allowing the classifier to generalize more effectively to unseen classes.

However, class-imbalanced malicious traffic brings challenge to both seen and unseen classifiers. Despite the strong performance in Micro Recall, XGBoost's average Macro Recall of 0.9188 and 0.9843 is lower than Micro Recall. This discrepancy indicates that while XGBoost is effective at classifying the majority classes, it struggles with minority classes, contributing to the lower Macro Recall. Interestingly, the unseen classifiers exhibit a different trend, with average Macro Recall being higher than average Micro Recall. The four models achieve average Macro Recall values of 0.7228, 0.7181, 0.6765, and 0.7525, while their corresponding Micro Recall values are 0.6786, 0.6909, 0.6884, and 0.7092, suggesting that unseen classifiers are more effective at recalling minority classes. This phenomenon can be attributed to the training strategy of the proposed SG-ZSL approach, where the generator synthesizes an equal number of samples for each class, and the unseen classifier is trained with a balanced synthesized dataset afterward.

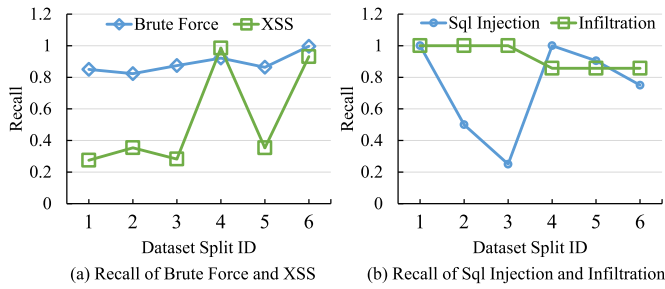


Fig. 14. Recall of four malicious traffic classes on different dataset splits. (a) Recall of Brute Force and XSS on six dataset splits. (b) Recall of Sql Injection and Infiltration on six dataset splits.

To investigate the impact of class imbalance on classifiers further, we utilize XGBoost as the seen classifier and the proposed SG-ZSL as the unseen classifier. As shown in Table VI, the results indicate that the unseen classifier is more adaptive to the class-imbalanced dataset. For instance, when Sql Injection, Infiltration, and PortScan are split as unseen malicious classes, their recall is 0.8918, 0.7778, and 0.4545, respectively. Despite Sql Injection and Infiltration only containing 21 and 36 samples, their recall is higher than other majority classes, suggesting that the unseen classifier trained with zero-shot learning is more conducive to achieving balanced classification results.

In contrast, XGBoost's Macro Recall is lower than Micro Recall, which is attributed to the influence of minority classes, similar to the discriminator. As illustrated in Figure 14(a), when Brute Force and XSS are split as seen classes, like dataset split ID 1, 2, 3, 5, both exhibit a decline in recall, with XSS showing a more significant drop. The decline indicates that these two classes share similar, indiscriminate features, leading to confusion of the classifier between these two classes. Moreover, due to XSS's smaller sample size than Brute Force, the model is more prone to misclassifying XSS samples as Brute Force. Similarly, Figure 14(b) shows that the recall for Infiltration and SQL Injection are suboptimal. The limited sample size of these classes leads classifiers to give biased predictions toward majority classes, resulting in recall below the average for these categories.

D. Hybrid Performance

To validate the feasibility and superiority of the AUF framework, we cascade the detector, discriminator, seen classifier, and unseen classifier and calculate the class-wise recall of the hybrid model. As shown in Table VI, despite facing the dual challenges of class imbalance and invisible malicious traffic, the hybrid model achieves a micro recall of 0.8601 and a macro recall of 0.8225 on the CIC-IDS2017 dataset. The hybrid model performs better on the BoT-IoT dataset, attaining a micro recall of 0.9828 and a macro recall of 0.9008. The superior results achieved by the hybrid model demonstrate that the proposed unified framework can accurately detect, discriminate, and classify malicious traffic while providing accurate prediction. When cascading the customized model designed to assist in labeling unseen malicious traffic, the hybrid model still achieves a micro recall of 0.8242 and a macro recall

of 0.7878 on the CIC-IDS2017 dataset. Considering that the detector and classifiers are raw XGBoost without the aid of any tricks, we are convinced that employing more powerful detectors and classifiers could significantly enhance overall performance.

In contrast, the cascaded model of HIoT and Verkerken et al.'s method exhibit suboptimal performance. On the CIC-IDS2017 dataset, HIoT achieves a micro recall of 0.9070 and a macro recall of 0.6259, while Verkerken et al.'s method is 0.8272 and 0.4700. Both methods are lower than the AUF framework. On the BoT-IoT dataset with fewer classes, the two methods perform even worse, attaining a micro recall of 0.7376, 0.7319 and a macro recall of 0.5409, 0.5149, respectively. Given that the AUF framework and the two comparison methods utilize identical detectors and classifiers, the lower recall can be attributed to a performance bottleneck caused by the suboptimal discriminator. As mentioned before, the two comparison methods wrongly discriminate a significant amount of seen malicious traffic as unseen malicious traffic while also misjudging a large portion of unseen malicious traffic as seen malicious traffic. The final classification results remain incorrect even if the classifier performs well. Verkerken et al.'s method improves HIoT by introducing an additional threshold selection stage aimed at recovering benign traffic that had been mistakenly identified as previously unseen malicious traffic. However, the added stage brings a negative effect on overall performance, rendering the improved approach less effective than the original HIoT. The primary reason lies in the relatively low detection accuracy of the autoencoder employed by Verkerken et al. Due to the autoencoder's limited performance, a significant portion of benign traffic is misclassified as unseen malicious traffic. Under such circumstances, the additional threshold selection stage can improve the recall of benign traffic. In contrast, XGBoost, which is trained in a supervised manner, can more accurately differentiate between benign and malicious traffic than the autoencoder. As a result, when using XGBoost, the proportion of benign traffic within the detected unseen malicious traffic is lower. Consequently, introducing an additional thresholding stage in this context has a negative effect. Given this, we only report the performance of HIoT in the subsequent confusion matrices.

The confusion matrices in Figure 15 further illustrate the differences between the AUF framework and HIoT, where the red labeling classes represent unseen classes and the black labeling classes represent seen classes. As shown in Figure 15 (a) and (c), owing to the excellent performance of the detector, discriminator, and classifiers, the AUF framework exhibits a clear diagonal pattern in both confusion matrices. For the CIC-IDS2017 dataset, the AUF framework achieves a recall rate above 0.85 across 13 classes. Only two classes exhibit lower recall: the BoT class and the XSS class. Due to the limited sample size of the BoT class, some samples are identified as benign traffic by the detector. Meanwhile, due to its feature similarity with the Brute Force class, some samples from the XSS class are misclassified as Brute Force by the classifier. The AUF framework performs better on the BoT-IoT dataset. Except for a small portion of samples from the

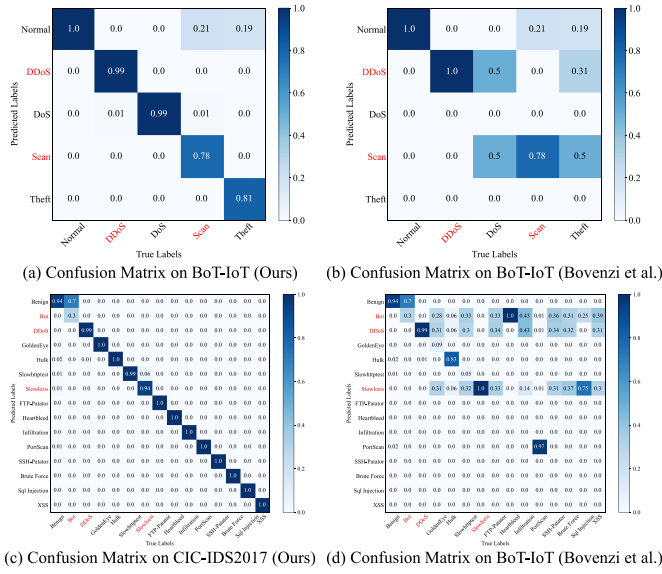


Fig. 15. The confusion matrix of the proposed AUF framework and HIoT. (a) The confusion matrix of the proposed AUF on the BoT-IoT dataset. (b) The confusion matrix of the HIoT on the BoT-IoT dataset. (c) The confusion matrix of the proposed AUF on the CIC-IDS2017 dataset. (d) The confusion matrix of the HIoT on the CIC-IDS2017 dataset.

Scan and Theft classes, which are recognized as the Normal class, most samples are correctly classified into their respective categories.

As depicted in Figure 15 (b) and (d), the performance disparity between HIoT and the AUF framework is pronounced. On the CIC-IDS2017 dataset, seven classes achieve a recall rate of 0. Only five classes—Benign, DDoS, PortScan, Hulk, and Slowloris—attain recall rates above 0.8. The confusion matrix reveals that the discriminator misjudges most seen malicious traffic as unseen malicious traffic, resulting in poor overall recall performance. This issue is even more evident in the BoT-IoT dataset, where nearly all visible malicious traffic categories are recognized as unseen malicious traffic. These results collectively indicate that HIoT performs poorly in distinguishing between seen and unseen malicious traffic, leading to significant deviations between the final classification results and the ground truth.

Based on above analysis, we further discuss the difference between AUF framework and HIoT [19]. Bovenzi et al.'s (GLOBECOM 2020) approach [19] achieves characteristics similar to AUF framework. Specifically, Bovenzi et al. deploy the anomaly detector before the classifier, enabling fast inference speed and distributed deployment. At the same time, Bovenzi et al. utilize the output of the classifier to discriminate between seen and unseen malicious traffic, enabling functionality completeness. However, we still categorize it as “non-framework cascading”, as simply deploying the anomaly detector prior to the misuse detector can hardly solve the “loss-gain imbalance” due to two key issues. On the one hand, the malicious traffic detected by the detector includes both seen and unseen categories, whereas the classifier can only classify seen malicious traffic. Without a mechanism to distinguish unseen malicious traffic based on the outputs of the detector or

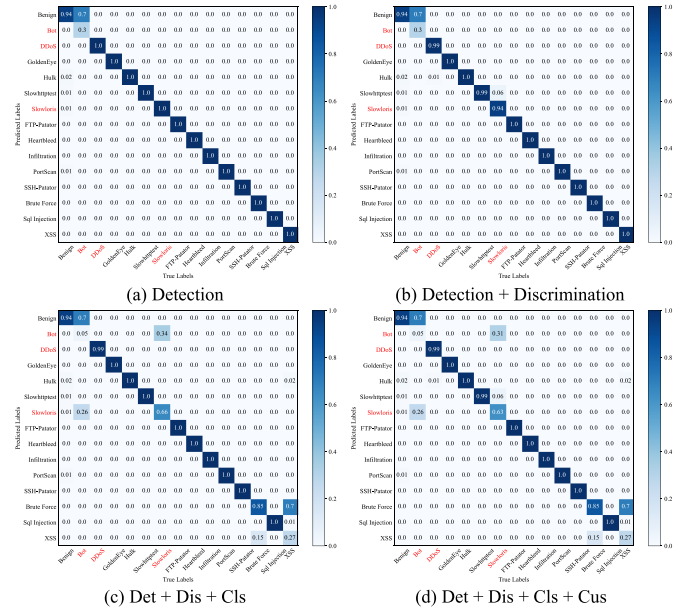


Fig. 16. The confusion matrix on evaluating different models in a cascaded manner, where red labeling represents the unseen malicious classes. (a) The confusion matrix of the detector. (b) The confusion matrix of cascaded detector and discriminator. (c) The confusion matrix of the cascaded detector, discriminator, and seen classifier. (d) The confusion matrix of the cascaded detector, discriminator, seen classifier, and customized model (unseen classifier).

classifier, the hybrid model cannot address unseen malicious traffic effectively.

On the other hand, even if unseen malicious traffic is identified based on the output of the detector or classifier, accurate prediction remains challenging. The detector and classifier are primarily designed to differentiate between benign and malicious traffic and to classify seen malicious traffic, rather than to identify unseen malicious traffic. Consequently, their outputs contain limited information for differentiating seen from unseen malicious traffic, leading to poor discrimination performance (e.g., Macro Recall of only 0.4994 on the BoT-IoT dataset, Table V). Inaccurate discrimination negatively impacts the overall prediction accuracy of the hybrid model (e.g., Macro Recall of 0.5409 on the BoT-IoT dataset, Table VI), ultimately rendering the hybrid model impractical.

As mentioned in Section II, the AUF framework achieves “unification” in three key aspects, especially in decoupling the discriminator as an independent component. The discriminator is essential for addressing the “loss-gain imbalance” problem, as it enables the hybrid model to perform discrimination and enhances overall prediction accuracy. On the one hand, the discriminator naturally bridges the detector and the classifier. It distinguishes seen and unseen malicious traffic from the detector’s output, forwarding only seen malicious traffic to the classifier for further classification. On the other hand, the discriminator is specifically designed to differentiate between seen and unseen malicious traffic. It accurately identifies unseen malicious traffic by thoroughly analyzing the differences in feature distributions and other characteristics (e.g., Micro Recall of 0.9926 and Macro Recall of 0.9345 on the BoT-IoT dataset, Table V). Additionally, the decoupling design enables more advanced discrimination methods and models to be integrated into the AUF framework and further

improve prediction accuracy without affecting the detector or classifier.

Most methods ignore some other performance metrics to improve prediction accuracy. The reason is that the core task of an HNIDS is to detect malicious traffic and identify its category accurately. Therefore, prediction accuracy is the most critical performance metric for ensuring the system's usability. We thoroughly compare the AUF framework with Bovenzi et al.'s approach and Verkerken et al.'s approach under identical experimental setups. The results indicate that AUF achieves higher prediction accuracy on both datasets (e.g., Macro Recall of 0.8225 versus 0.4717 in the CIC-IDS2017 dataset and Micro Recall of 0.9008 versus 0.5409 in the BoT-IoT dataset, Table VI), demonstrating its superiority and usability.

In summary, the AUF is innovative in its design and resolution of the "loss-gain imbalance," setting it apart from prior works. The AUF framework not only overcomes the limitations of the "Non-framework cascading" but also provides new insights for future HNIDS development.

As mentioned before, the performance of downstream models is closely related to the effectiveness of upstream models for cascaded models. For example, the average recall rate of the Heartbleed is 0.8033, 0.8692, 0.9156, and 0.9437 at the detection, discrimination, classification, and customized stages, respectively. The hybrid recall for the Heartbleed class is only 0.7041, indicating that the detector's suboptimal performance on Heartbleed hinders the overall performance of the hybrid model. To illustrate this phenomenon more clearly, we plot the confusion matrices of the cascaded model at different stages on the CIC-IDS2017 dataset. Figure 16 shows the confusion matrices on evaluating different models in a cascaded manner. We observe that most malicious traffic is detected during the detection stage, with only a few malicious samples from the Bot being wrongly recognized as benign traffic. The discriminator correctly identifies most of the detected malicious traffic afterward, with only a few malicious traffic from Infiltration being wrongly discriminated. The classifier correctly classifies most of the samples identified as seen malicious classes, except those belonging to the BruteForce and XSS classes. As previously analyzed, the BruteForce and XSS samples tend to be confused by the classifier due to their similar features. After introducing the customized unseen class classifier, most samples from unseen malicious classes are correctly labeled. Despite the strong performance of all four models, there are still discrepancies between the final confusion matrix and the one from the detection stage. Therefore, improving the performance of any of the detector, discriminator, classifier, or customized models would enhance the final performance of the hybrid model.

We also notice that the model's generalization to the minority classes is crucial. Apart from the discriminator, we do not adopt other strategies to mitigate the effects of class imbalance to reveal its impact on the model. The results indicate whether the detector, discriminator, or classifier tends to give predictions biased towards the majority classes, resulting in recall for the minority classes being below average. This phenomenon is exponentially amplified in the cascaded

model. For example, the average recall of the XSS drops sharply from 0.9942 at the detection stage to 0.3976 after cascading. Therefore, designing training strategies for class-imbalanced datasets to enable the model to provide unbiased predictions is critical.

VI. CONCLUSION

This article explores the limitations of the "Non-framework cascading" and the "MADF" framework, thereby introducing a unified framework AUF to address these issues. The key difference between the AUF framework and other HNIDSs is that it consists of four cascaded and decoupled models, each performing a corresponding functionality: distinguishing between benign and malicious traffic, differentiating between seen and unseen malicious traffic, identifying the category of seen malicious traffic, and implementing customized functionalities. This design overcomes the "loss-gain imbalance" issue, thereby providing more comprehensive functionalities, faster inference speed, and higher accuracy. In addition, the unified framework achieves the unification of three aspects, offering a new perspective for the design of future HNIDS.

With the AUF framework, we validate the performance of individual and cascaded models across the CIC-IDS2017 and the BoT-IoT datasets, including the adaptive KNN-based discriminator and the customized model SG-ZSL proposed in this article. Extensive experiments demonstrate the feasibility and superiority of the AUF framework. Our findings also suggest that: (1) The dataset split proposed in this article includes seen and unseen malicious traffic together, which provides new evaluation criteria and baselines for future HNIDSs. (2) Designing detectors that can accurately discover unseen malicious traffic is critical for improving HNIDS performance. (3) Introducing detectors, discriminators, and classifiers with more robust generalization to minority classes in future work will further enhance the overall performance of the HNIDS. (4) Integrating flow-based network intrusion detection methods into the AUF framework will explore the potential in online malicious traffic detection.

REFERENCES

- [1] D. Chou and M. Jiang, "A survey on data-driven network intrusion detection," *ACM Comput. Surv.*, vol. 54, no. 9, p. 182, Oct. 2021.
- [2] K. He, D. D. Kim, and M. R. Asghar, "Adversarial machine learning for network intrusion detection systems: A comprehensive survey," *IEEE IEEE Commun. Surveys Tuts.*, vol. 25, no. 1, pp. 538–566, 1st Quart., 2023.
- [3] G. Pang, C. Shen, L. Cao, and A. V. D. Hengel, "Deep learning for anomaly detection: A review," *ACM Comput. Surveys*, vol. 54, no. 2, pp. 1–38, 2021.
- [4] A. Thakkar and R. Lohiya, "A survey on intrusion detection system: Feature selection, model, performance measures, application perspective, challenges, and future research directions," *Artif. Intell. Rev.*, vol. 55, no. 1, pp. 453–563, Jan. 2022.
- [5] Y. Wang et al., "Adversarial attacks and defenses in machine learning-empowered communication systems and networks: A contemporary survey," *IEEE IEEE Commun. Surveys Tuts.*, vol. 25, no. 4, pp. 2245–2298, 4th Quart., 2023.
- [6] Y. Guo, "A review of machine learning-based zero-day attack detection: Challenges and future directions," *Comput. Commun.*, vol. 198, pp. 175–185, Jan. 2023.
- [7] M. Ozkan-Okay, R. Samet, Ö. Aslan, and D. Gupta, "A comprehensive systematic literature review on intrusion detection systems," *IEEE Access*, vol. 9, pp. 157727–157760, 2021.

- [8] U. Sabeel, S. S. Heydari, K. El-Khatib, and K. Elgazzar, "Unknown, atypical and polymorphic network intrusion detection: A systematic survey," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 1, pp. 1190–1212, Feb. 2024.
- [9] E. M. Maseno, Z. Wang, H. Xing, and L. Maglaras, "A systematic review on hybrid intrusion detection system," *Sec. Commun. Netw.*, vol. 2022, pp. 1–23, Jan. 2022.
- [10] R. Ahmad, I. Alsmadi, W. Alhamdani, and L. Tawalbeh, "Zero-day attack detection: A systematic literature review," *Artif. Intell. Rev.*, vol. 56, no. 10, pp. 10733–10811, Feb. 2023.
- [11] K. N. Rao, K. V. Rao, and P. V. G. D. Prasad Reddy, "A hybrid intrusion detection system based on sparse autoencoder and deep neural network," *Comput. Commun.*, vol. 180, pp. 77–88, Dec. 2021.
- [12] G. Kim, S. Lee, and S. Kim, "A novel hybrid intrusion detection method integrating anomaly detection with misuse detection," *Expert Syst. Appl.*, vol. 41, no. 4, pp. 1690–1700, 2014.
- [13] M. S. ElSayed, N.-A. Le-Khac, M. A. Albahar, and A. Jurcut, "A novel hybrid model for intrusion detection systems in SDNs based on CNN and a new regularization technique," *J. Netw. Comput. Appl.*, vol. 191, Oct. 2021, Art. no. 103160.
- [14] E. Mushtaq, A. Zameer, M. Umer, and A. A. Abbasi, "A two-stage intrusion detection system with auto-encoder and LSTMs," *Appl. Soft Comput.*, vol. 121, May 2022, Art. no. 108768.
- [15] J. Lan, X. Liu, B. Li, and J. Zhao, "A novel hierarchical attention-based triplet network with unsupervised domain adaptation for network intrusion detection," *Appl. Intell.*, vol. 53, no. 10, pp. 11705–11726, Sep. 2022.
- [16] C. A. de Souza, C. B. Westphall, R. B. Machado, J. B. M. Sobral, and G. dos Santos Vieira, "Hybrid approach to intrusion detection in fog-based IoT environments," *Comput. Netw.*, vol. 180, Oct. 2020, Art. no. 107417.
- [17] Z. Chen, M. Simsek, B. Kantarci, M. Bagheri, and P. Djukic, "Machine learning-enabled hybrid intrusion detection system with host data transformation and an advanced two-stage classifier," *Comput. Netw.*, vol. 250, Aug. 2024, Art. no. 110576.
- [18] Y. An, Y. He, F. R. Yu, J. Li, J. Chen, and V. C. M. Leung, "An HTTP anomaly detection architecture based on the Internet of intelligence," *IEEE Trans. Cogn. Commun. Netw.*, vol. 8, no. 3, pp. 1552–1565, Sep. 2022.
- [19] G. Bovenzi, G. Aceto, D. Ciunzo, V. Persico, and A. Pescapé, "A hierarchical hybrid intrusion detection approach in IoT scenarios," in *Proc. IEEE Global Commun. Conf.*, Dec 2020, pp. 1–7.
- [20] M. Verkerken et al., "A novel multi-stage approach for hierarchical intrusion detection," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 3, pp. 3915–3929, Sep. 2023.
- [21] H. Kye, M. Kim, and M. Kwon, "Hierarchical detection of network anomalies: A self-supervised learning approach," *IEEE Signal Process. Lett.*, vol. 29, pp. 1908–1912, 2022.
- [22] A. Alzaqebah, I. Aljarah, and O. Al-Kadi, "A hierarchical intrusion detection system based on extreme learning machine and nature-inspired optimization," *Comput. Security*, vol. 124, Jan. 2023, Art. no. 102957.
- [23] N. Wei et al., "An autoencoder-based hybrid detection model for intrusion detection with small-sample problem," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 2, pp. 2402–2412, Apr. 2024.
- [24] J. Yang, X. Chen, S. Chen, X. Jiang, and X. Tan, "Conditional variational auto-encoder and extreme value theory aided two-stage learning approach for intelligent fine-grained known/unknown intrusion detection," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 3538–3553, 2021.
- [25] S. Li, Y. Cao, S. Liu, Y. Lai, Y. Zhu, and N. Ahmad, "HDA-IDS: A hybrid dos attacks intrusion detection system for IoT by using semi-supervised CL-GAN," *Expert Syst. Appl.*, vol. 238, Mar. 2024, Art. no. 122198.
- [26] L. Yang, A. Moubayed, and A. Shami, "MTH-IDS: A multitiered hybrid intrusion detection system for Internet of Vehicles," *IEEE Internet Things J.*, vol. 9, no. 1, pp. 616–632, Jan. 2022.
- [27] R. Chen, L. Luo, X. Wang, B. Ren, D. Guo, and S. Zhu, "Knowing the unknowns: Network traffic detection with open-set semi-supervised learning," *Comput. Netw.*, vol. 251, Sep. 2024, Art. no. 110630.
- [28] Y. Zhong, Z. Wang, X. Shi, J. Yang, and K. Li, "RFG-HELAD: A robust fine-grained network traffic anomaly detection model based on heterogeneous ensemble learning," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 5895–5910, 2024.
- [29] Y. Sun, Y. Ming, X. Zhu, and Y. Li, "Out-of-distribution detection with deep nearest neighbors," in *Proc. 39th Int. Conf. Mach. Learn.*, vol. 162, Jul. 2022, pp. 20827–20840.
- [30] Z. Han, Z. Fu, S. Chen, and J. Yang, "Contrastive embedding for generalized zero-shot learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2021, pp. 2371–2381.
- [31] Q. Li, Z. Zhan, Y. Shen, and B. Bhanu, "Co-GZSL: Feature contrastive optimization for generalized zero-shot learning," *Neural Process. Lett.*, vol. 56, no. 2, pp. 1–16, 2024.
- [32] W. Fan, C. Liang, and T. Wang, "Contrastive semantic disentanglement in latent space for generalized zero-shot learning," *Knowl.-Based Syst.*, vol. 257, Dec. 2022, Art. no. 109949.
- [33] H. Wang, T. Zhang, and X. Zhang, "Contrastive embedding-based feature generation for generalized zero-shot learning," *Int. J. Mach. Learn. Cybern.*, vol. 14, no. 5, pp. 1669–1681, 2023.
- [34] X. Kong et al., "En-compactness: Self-distillation embedding & contrastive generation for generalized zero-shot learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022, pp. 9296–9305.
- [35] E. Aghaei, X. Niu, W. Shadid, and E. Al-Shaer, "SecureBERT: A domain-specific language model for cybersecurity," in *Security and Privacy in Communication Networks*, F. Li, K. Liang, Z. Lin, and S. K. Katsikas, Eds., Cham, Switzerland: Springer, 2023, pp. 39–56.
- [36] A. v. d. Oord, Y. Li, and O. Vinyals, "Representation learning with contrastive predictive coding," 2018, *arXiv:1807.03748*.
- [37] N. Koroniotis, N. Moustafa, E. Sitnikova, and B. Turnbull, "Towards the development of realistic botnet dataset in the Internet of Things for network forensic analytics: Bot-IoT dataset," *Future Gener. Comput. Syst.*, vol. 100, pp. 779–796, Nov. 2019.
- [38] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Inf. Syst. Security Privacy*, 2018, pp. 108–116.
- [39] V. Bulavas, V. Marcinkevicius, and J. Rumiński, "Study of multi-class classification algorithms' performance on highly imbalanced network intrusion datasets," *Informatica*, vol. 32, no. 3, pp. 441–475, 2021.
- [40] S. Han, X. Hu, H. Huang, M. Jiang, and Y. Zhao, "AdBench: Anomaly detection benchmark," in *Advances in Neural Information Processing Systems*, S. Koyejo et al., Eds., vol. 35, Cambridge, MA, USA: MIT Press, 2022, pp. 32142–32159.
- [41] Z. Li, Y. Zhu, and M. Van Leeuwen, "A survey on explainable anomaly detection," *ACM Trans. Knowl. Disc. Data*, vol. 18, no. 1, pp. 1–54, 2023.
- [42] T. Zebin, S. Rezvy, and Y. Luo, "An explainable AI-based intrusion detection system for DNS over HTTPS (DoH) attacks," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 2339–2349, 2022.
- [43] L. Yang, S. Fu, Y. Wang, L. Liu, and Y. Luo, "The analysis of encrypted video stream based on low-dimensional embedding method," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 8280–8295, 2025.
- [44] S. Cai, H. Tang, J. Chen, T. Lv, W. Zhao, and C. Huang, "GSA-DT: A malicious traffic detection model based on graph self-attention network and decision tree," *IEEE Trans. Netw. Service Manag.*, vol. 22, no. 2, pp. 2059–2073, Apr. 2025.
- [45] O. Aouedi, K. Piamrat, and B. Parrein, "Ensemble-based deep learning model for network traffic classification," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 4, pp. 4124–4135, Dec. 2022.
- [46] D. Upadhyay, J. Manero, M. Zaman, and S. Sampalli, "Intrusion detection in SCADA based power grids: Recursive feature elimination model with majority vote ensemble algorithm," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 3, pp. 2559–2574, Jul.–Sep. 2021.
- [47] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *Proc. 25th Annu. Netw. Distrib. Syst. Security Symp.*, San Diego, CA, USA, 2018, pp. 1–15.
- [48] G. Bovenzi, G. Aceto, D. Ciunzo, A. Montieri, V. Persico, and A. Pescapé, "Network anomaly detection methods in IoT environments via deep learning: A fair comparison of performance and robustness," *Comput. Security*, vol. 128, May 2023, Art. no. 103167.



Yan Wang received the B.S. degree from Yunnan University, Kunming, China. He is currently pursuing the M.S. degree with the School of Computer Science and Engineering, University of Electronic Science and Technology of China, Chengdu, China. His current research interests include deep learning and intrusion detection.



Sheng Cao (Member, IEEE) received the Ph.D. degree from the University of Chinese Academy Sciences, Beijing, in 2008. He is currently a Professor with the University of Electronic Science and Technology of China, Chengdu. His research interests include network security, blockchain, service computing, and intelligent education.



Xiaosong Zhang (Senior Member, IEEE) received the M.S. and Ph.D. degrees from the University of Electronic Science and Technology of China (UESTC), Chengdu, China, in 1999 and 2011, respectively. He is currently a Professor with the School of Computer Science and Engineering, UESTC. He is the Cheung Kong Scholar Distinguished Professor. He is also the Head of College for Cyber Security, UESTC. His research interests are blockchain and AI security.



Jingwei Li (Member, IEEE) received the B.S. degree in information and computing science from the Hebei University of Technology, Tianjin, China, in 2009, and the Ph.D. degree in computer application technology from Nankai University, Tianjin, in 2014. Since 2016, he has been an Associate Professor with the University of Electronic Science and Technology of China, Chengdu, China. His current research interests include applied cryptography, and cloud security and secure deduplication.